

Data Visualization with Python

Abstract

One of the most important skills of successful data scientists and data analysts is the ability to **tell a compelling story by visualizing data and findings** in an approachable and stimulating way. In this course you will learn many ways to effectively visualize both small and large-scale data. You will be able to take data that at first glance has little meaning and present that data in a form that conveys insights.

This course will teach you to work with many Data Visualization tools and techniques. You will learn to create various types of basic and advanced graphs and charts like: Waffle Charts, Area Plots, Histograms, Bar Charts, Pie Charts, Scatter Plots, Word Clouds, Choropleth Maps, and many more! You will also create interactive dashboards that allow even those without any Data Science experience to better understand data and make more effective and informed decisions.

You will learn hands-on by completing numerous labs and a final project to practice and apply the many aspects and techniques of Data Visualization using Jupyter Notebooks and a Cloud-based IDE. You will use several data visualization libraries in Python, including Matplotlib, Seaborn, Folium, Plotly & Dash.

Course Learning Objectives

After completing this course, a learner will be able to:

- Implement data visualization techniques and plots using Python libraries, such as Matplotlib, Seaborn, and Folium to tell a stimulating story.
- Create different types of charts and plots such as line, area, histograms, bar, pie, box, scatter, and bubble.
- Create advanced visualizations such as waffle charts, word clouds, regression plots, maps with markers, & choropleth maps.
- Generate interactive dashboards containing scatter, line, bar, bubble, pie, and sunburst charts using the Dash framework and Plotly library.

Module 1

Title: Introduction to Data Visualization Tools

Description

Data visualization is a way of presenting complex data in a form that is graphical and easy to understand. When analyzing large volumes of data and making data-driven decisions, data visualization is crucial. In this module, you will learn about data visualization and some key best practices to follow when creating plots and visuals. You will discover the history and the architecture of Matplotlib. Furthermore, you will learn about basic plotting with Matplotlib and explore the dataset on Canadian immigration, which you will use during the course. Lastly, you will analyze data in a data frame and generate line plots using Matplotlib.

Objectives

By the end of this week, you will be able to:

- Discuss data visualization and its importance
- Discover the history of Matplotlib and its architecture
- Use Matplotlib to create plots employing Jupyter notebook
- Explore the dataset on immigration to Canada
- Identify the steps to analyze data in Pandas data frame
- Use Matplotlib to create line plots

Activities

Lesson 0: Welcome to the Course

Welcome to the data visualization with Python course.

The data visualization with Python course is ideal for **professionals with knowledge of Python** programming language who aspire to **become data analysts**.

In this course, you'll learn:

- How to create meaningful, effective and aesthetically pleasing data visuals and plots in Python using Matplotlib.
- You'll explore advanced visualization tools that provide various advanced features and capabilities.

- In addition, you'll identify different **web-based dashboarding tools** available in Python.
- The last module will allow you to act as a data analyst to complete your final lab assignment.

What you will learn

			
Create impactful data visuals	Explore advanced visualization tools	Identify different web-based dashboarding tools	Practice data visualization skills

The five modules in this course comprehensively cover the concepts related to these areas.

In **module 1**, you will learn about data visualization and best practices for creating plots and visuals, you will discover the history and **architecture of Matplotlib**. Furthermore, you'll learn about basic plotting with Matplotlib and explore the dataset on Canadian immigration, which you will use during the course. Lastly, you will analyze data in a DataFrame and generate line plots using Matplotlib.

Module 1: Introduction to Data Visualization

- Best practices
- History and architecture of Matplotlib
- Basic plotting with Matplotlib
- Dataset on Canadian immigration
- Data analysis



In **module 2**, you will create a diverse range of plots using Matplotlib and the data visualization library. This module will teach you about area plots, histograms, bar charts, pie

charts, box plots, and scatter plots. You'll also explore the process of creating these visualization tools using Matplotlib.

Module 2: Basic and Specialized Visualization Tools

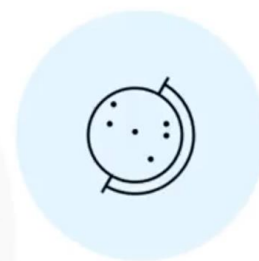
- Diverse range of plots
 - Area plots,
 - Histogram
 - Bar charts
 - Pie charts
 - Box plots
 - Scatter plots
- Process of creating visualization tools using Matplotlib



In **module 3**, you'll learn about waffle charts and word cloud. You will create regression plots using seaborn, a new visualization library in Python. In addition, you'll learn about **Folium**, a data visualization library that visualizes geospatial data. Furthermore, you'll explore the process of creating maps using folium and super imposing them with markers to make them interesting. Finally, you'll learn how to create a choropleth map using folium.

Module 3: Advanced Visualizations and Geospatial Data

- Waffle charts and word cloud
- Seaborn
- Folium
- Process of creating maps using Folium
- Creating Choropleth map using Folium



In **module 4**, you'll explore the benefits of dashboards and identify the different web-based dashboarding tools in Python. You will learn about Plotly and discover how to use Plotly graph objects and Plotly express to create charts, you will gain insight into Dash and open source user interface, Python library and its two components. Finally, you will clearly

understand the callback function and determine how to connect core and HTML components using callback.

Module 4: Creating Dashboards with Plotly and Dash

- Benefits of dashboards
- Different web-based dashboarding tools
- Plotly
 - Plotly graph objects
 - Plotly express
- Dash



Finally, **module 5**, primary focus is helping you complete your final lab assignment. Your objective is to track and report on the performance of domestic flights operated by US airlines. The objective is to analyze the reporting airlines performance to improve flight and customer reliability.

The objective is to analyze automobile sales data covering periods of recession and not recession.

Module 5: Final Project and Exam

- Final lab assignment
- Analyze the automobile sales data



After completing this course, you can use different visualization libraries in Python, namely Matplotlib, seaborn, and folium, to create expressive visual representations of your data for different purposes. You'll also be able to use basic and advanced visualization tools and identify different web-based dashboarding tools.

Key takeaways

- Use different visualization libraries in Python
- Use basic and advanced visualization tools
- Identify different web-based dashboarding tools

To get the most from this course, make sure to view every video, review your knowledge with each quiz, and complete all the hands-on labs. If you have any trouble with the course material, please don't hesitate to contact us and the discussion forum.

Making the most out of the course

- View every video
- Review your knowledge with each quiz
- Complete all the hands-on labs

Lesson 1: Introduction to Data Visualization

Overview of Data Visualization

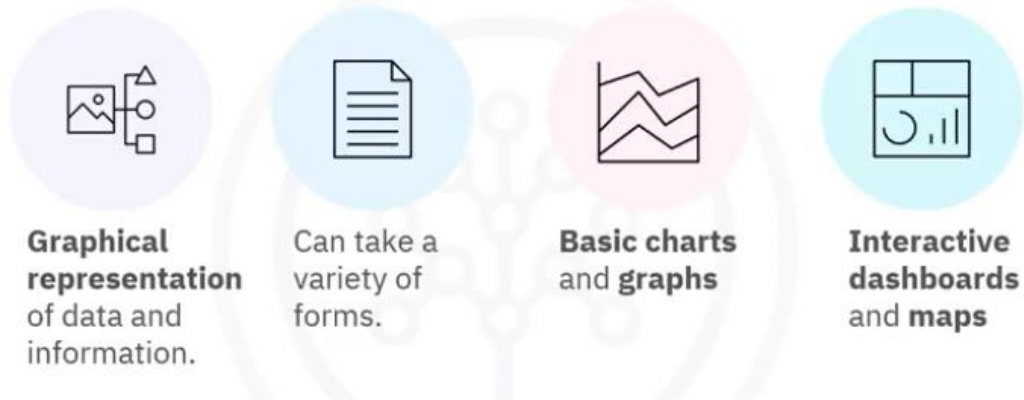
Hello, and welcome to overview of data visualization.

After watching this video, you'll be able to

- Explain what data visualization is and why it's important.
- Describe the uses for data visualization.

- Explore the best practices for data visualization.

What is Data Visualization?

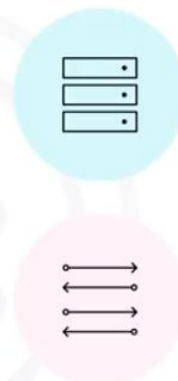


Data visualization is the **graphical representation** of data and information. It involves the process of creating visual representations of data. Data visualization can take **many different forms**, from basic charts and graphs to more complex interactive dashboards, maps, and infographics. Basic charts and graphs are the simplest form of data visualization to represent numerical data. **Interactive dashboards** and maps are more complex forms of visualization to provide real-time information.

We need data visualization for several reasons.

Why build visuals?

- Easily Understand Data
 - Highlights patterns, trends, and relationships
- Communicate Insights
 - Simplifies the data for stakeholders



Data visualization helps us to easily **understand complex datasets** that might be difficult to comprehend in their raw form. It can **highlight patterns, trends, and relationships** that might not be immediately apparent from looking at the data. Visualization enables us to

communicate insights and findings to stakeholders in a more compelling and understandable way.

It allows us to tell a story with data, making it **easier** for stakeholders **to understand** and act on the information presented.

Why build visuals?

- Identify trends and data patterns
 - Helps gain new insights
- Present data for better understanding and interpretation



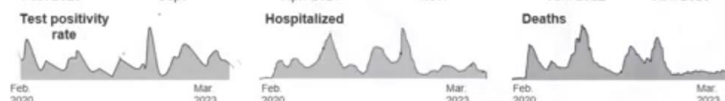
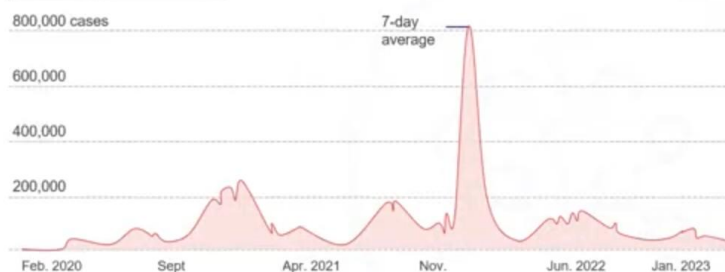
Visualization helps us **identify trends and data patterns** that might otherwise be difficult to see. We can gain new insights and make informed decisions by examining data over time or across different dimensions. Presenting data in a more easily understood and interpreted way enables us to identify opportunities, spot potential problems, and **make more informed choices**.

Let's explore how data visualization can help us understand data.

Leveraging the power of data visualization

New reported cases

All time Last 90 Days



Vaccinations

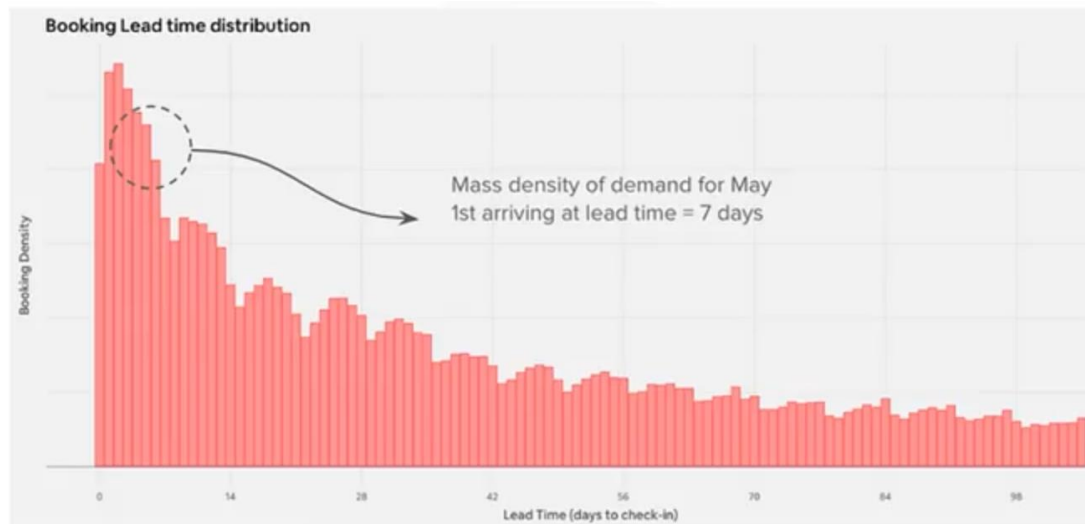
	FULLY VACCINATED	WITH A B@STER
All ages	68%	34%
65 and up	93%	67%

[See more details >](#)

[About this data](#)

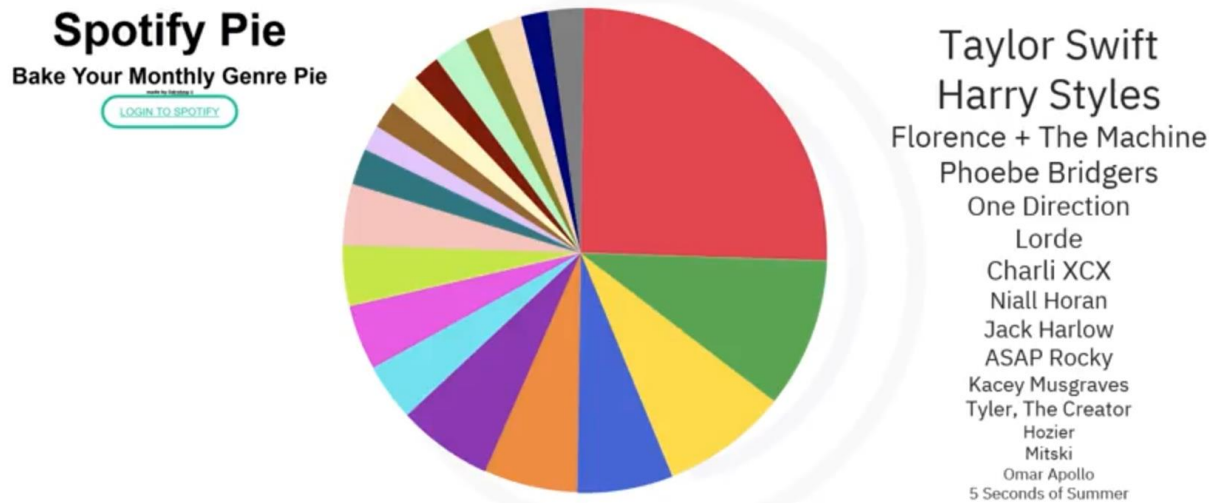
The New York Times, a well-known news publication, has used data visualization to help people understand the impact of the COVID-19 pandemic in the United States. The number of daily new reported cases and the number of deaths are shown through a line graph and the vaccinated population percentage is displayed using a bar graph.

Airbnb uses [smart pricing framework](#) to assist hosts and determining the best possible prices for their listings, thereby increasing their profits using a bar graph, the chart displays the lead time distribution aggregated to a market level.



Spotify uses [Spotify Pie](#), a unique data visualization tool that enables users to create customized pie charts based on their listening habits on Spotify. The chart displays the distribution of different genres, artists, and tracks the user has listened to.

Leveraging the power of data visualization



What you see here is a choropleth map on [Netflix's Insight Tool dashboard](#) with other visualizations like line graphs and bar plots. These dynamic dashboards help them gain a deeper understanding of operations, identify areas for improvement, and quickly address any issues.

Data visualization is an essential tool for anyone dealing with data.

Leveraging the power of data visualization

- Business and finance
- Healthcare
- Education
- Government
- Research and science
- Entertainment

- Companies can gain insights into market trends, financial performance, and customer behavior.
- Healthcare professionals can identify patterns in patient data and develop targeted treatment plans.
- Visualization helps analyze student performance and inform instructional decisions leading to better academic outcomes.
- Visualization helps governments make informed decisions and communicate data to the public.

- Visualization helps scientists and researchers analyze complex data, develop new insights, and share findings effectively.
- Visualization helps workers and the entertainment industry determined ratings and ultimately signals to the creators what the audience wants.

Although data visualization has become an essential tool, not all visualizations are created equal. To create **effective visualizations**, it's important to follow **best practices** that can help ensure that the **data is accurately represented** and the message is **clearly communicated**.

Best practices

- Data is accurately represented
- Message is clearly communicated

Best Practices:

- Choosing the **appropriate visualization type** for the presented data. Different types of data require different visualization techniques. Choose the right one to communicate that data effectively.
- Keep the visualization **simple** and easy to read. A line chart shows trends over time. Well, a bar chart is ideal for comparing values.
- To ensure that the data is accurately represented, it's also important to **label the axis** clearly and provide context for the data being presented. Include a clear **title**, summarizing the main message and a **legend** explaining the meaning of any symbols or colors used.
- Including **unnecessary graphics** or information, can confuse the audience and distract them from the main message. **Keep visualization focused** on the main point and use only the necessary data and labels.
- Lastly, it's important to **consider the audience** when creating the visualization. Who will view the visualization? What do they need to know? The visualization should be tailored to their needs and understanding.

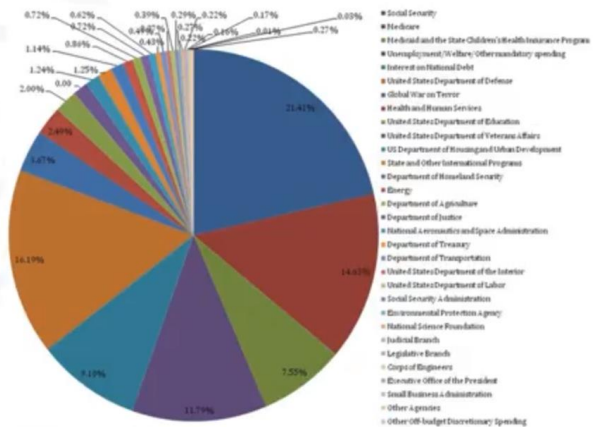
- 

-

- What is the data
and the information

-

- CE: <https://en.wikipedia.org>

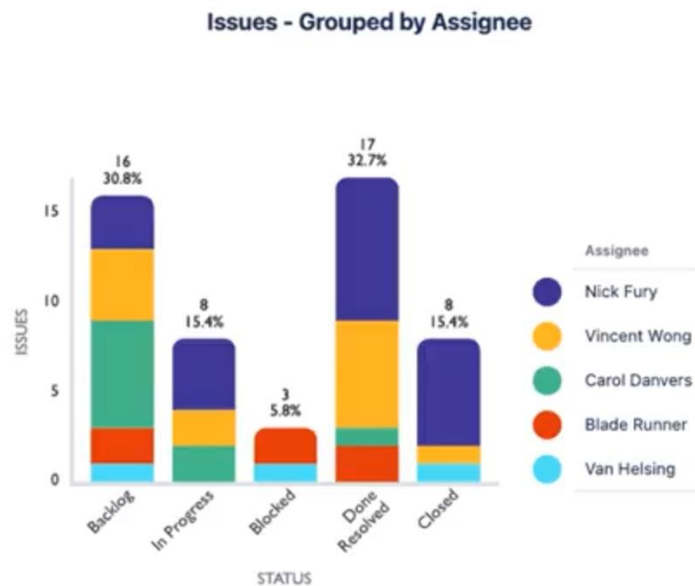


SOURCE: https://en.wikipedia.org/wiki/2007_United_States_federal_budget#.

This chart represents a team's board. The stacked bars depict the percentage of the work assigned to the individual users and their work progress. Colors are distinguishable and consistent. The legend shows the individual color's meanings.

The chart provides more clarity with the values and percentages on top of each bar.

Best practices (good example)



Darkhorse Analytics is a company that was created out of a research lab at the University of Alberta in 2008. The company has done fascinating work on data visualization.

[Darkhorse](#) Analytics specializes in quantitative consulting in several areas, including data visualization and geospatial analysis. Their approach when creating a visual revolves around the idea that **less is more effective**, more attractive, and more impactful. By following these best practices, data visualizations can become a powerful tool. Checkout the Darkhorse Analytics website for examples of good and bad plots.

In this video, you learned that **data visualization is the process of presenting data in a visual format**, such as charts, graphs, and maps **to help people understand and analyze data easily**.

Data visualization has diverse use cases such as business, science, health care, and finance.

It's important to follow best practices, such as **selecting appropriate visualizations** for the data being presented, choosing **colors and fonts that are easy to read** and interpret, and **minimizing clutter**

Recap

In this video, you learned that:

- Data visualization is the process of presenting data in a visual format
- Data visualization has diverse use cases, such as in business, science, healthcare, and finance.
- It is important to follow data visualization best practices for presenting data

Types of Plots

After watching this video, you'll be able to:

- explore different types of plots available for visualizing data.
- Identify the characteristics and appropriate use cases for each type of plot.

As we know, data visualization **represents data** using visual formats like graphs, charts and maps.

It effectively **communicates information**, trends and insights concisely, it also **uncovers patterns**, identify trends and simplifies complex information for **easy understanding**.

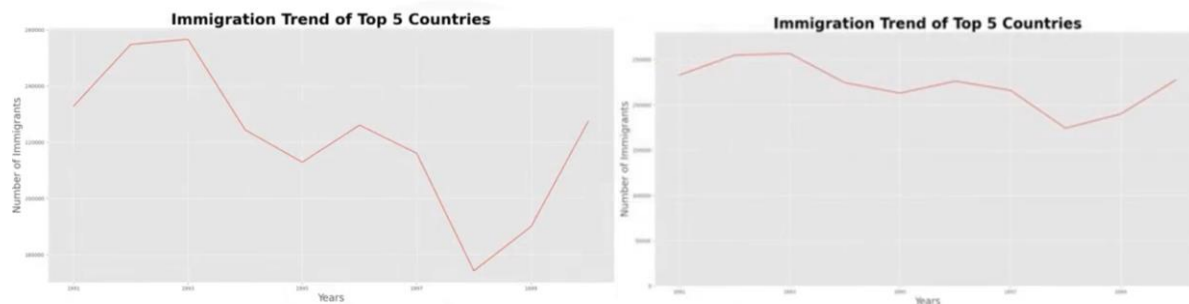
Line plot

A line plot, also known as a line chart, displays data as a series of data points connected by straight lines.

- **Display trends change over time**, such as stock market fluctuations or temperature.
- Compare datasets with a **continuous independent variable** like age or time.
- Illustrate **cause and effect relationships**, such as sales revenue changes based on marketing budget.
- Visualize **continuous data**, like height measurements over time.

Line plots can be **misleading if the scales on the axes are not carefully chosen** to reflect the data accurately.

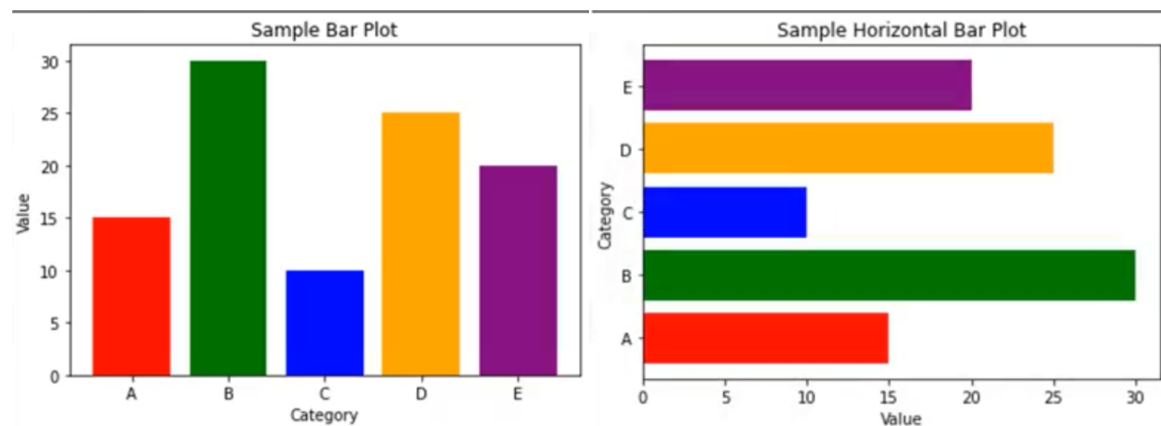
The line plot shows a downfall in immigration trends in 1998, however, if you will **correct the scale on the y-axis to start with zero**, as in this plot, you'll see that the actual trend is not that alarming.



Bar plot

A bar plot, also known as a bar chart, displays data using rectangular bars, where the **height or length** of the bars **represents** the **magnitude** of the data.

The bars can be oriented either vertically or horizontally.



A vertically oriented bar chart is often referred to as a **vertical bar chart** or a **column chart**.

If you're looking for an effective way to compare data, bar plots are ideal for:

Comparing different categories or groups.

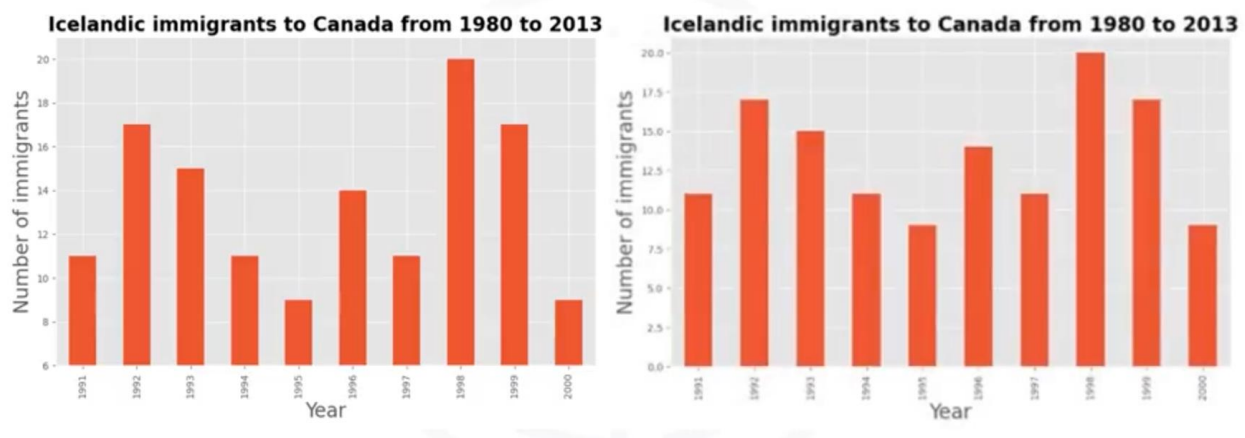
They excel with **discrete data**, like comparing sales revenue by product.

They show how different categories contribute to the whole and rankings, such as sales percentage or budget allocation.

Bar plots can visualize data that you can easily rank, like displaying the bestselling books in the market.

Their **simplicity and interpretability**, make them **avored** for data visualization.

Inaccurate bar choices or axis scales can lead to misleading plots.



In this bar plot, the y-axis starting at six, makes the difference between the values in 1995 and 1998 seem larger than it is. However, when the y-axis starts at zero, the plot accurately represents the data without misleading the viewer.

Scatter plot

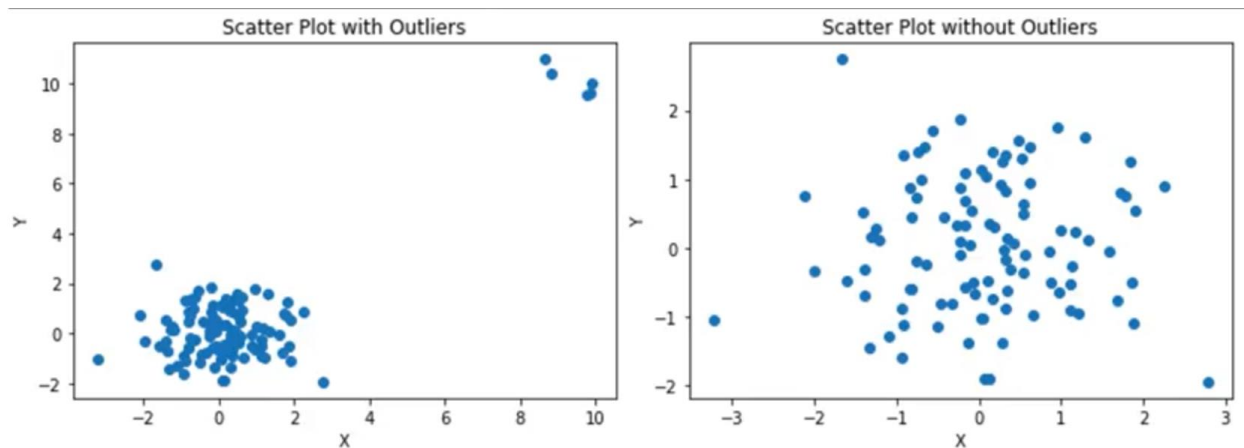
A scatter plot is a type of plot that presents values for **two variables** for a set of data using Cartesian coordinates.

The data points are displayed as a collection of points, where one variable's value determines the position on the horizontal axis and the other variable's value determines the position on the vertical axis.

You can use it for:

- Examining the **relationship between two continuous variables** like temperature and energy consumption.
- Investigating **patterns or trends in data**, such as house prices versus size.
- **Detecting outliers** or unusual observations such as outliers in test scores or abnormal stock behavior.
- Visualizing data with many observations to **identify clusters** or groups.
- Exploring complex data.

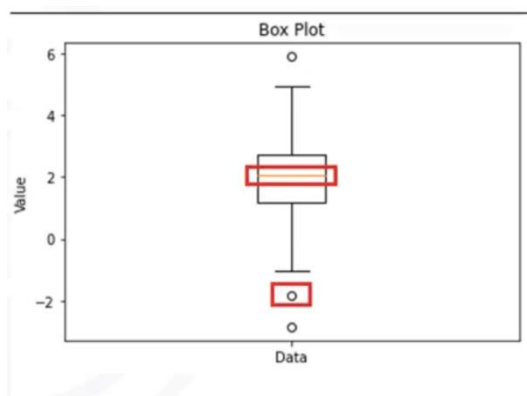
Outliers significantly impact interpretation, requiring consideration of their inclusion or exclusion.



On the screen, the data is plotted with and without outliers. With outliers, it shows two clusters, but removing outliers makes the remaining data more visible. **Proper outlier handling enhances accuracy** and meaningful insights in scatter plots.

Box plot.

A box plot, also known as a box and whisker plot, is a type of plot that displays the **distribution of a data set**, along with key statistical measures.



It consists of a box, representing the **interquartile range, IQR**, a line inside the box representing the median and lines whiskers extending from the box to indicate the range of the data, excluding outliers.

Outliers may be represented as individual data points beyond the whiskers.

Let's look at some scenarios where box plots come in handy.

- While **comparing the distribution of a continuous variable** across different categories or groups. For example, employees' salaries across departments.
- Examining **spread and skewness** of data set, visualizing quartiles and outliers
- Identifying and analyzing potential **outliers** within a data set.
- Visualizing **summary statistics**, median, quartiles, range in a concise and informative manner.
- **Comparing distributions of multiple variables** in datasets side by side.

Remember, box plots provide valuable information about outliers, such as their presence and extent. **Ignoring or mishandling outliers, can distort the interpretation of the data** and mask important insights, same as with scatter plots.

Histogram.

A histogram, is a graphical representation of the **distribution of a data set, showing the frequency or relative frequency of values within specific intervals.**

It consists of bars, where the height represents the **data count in each interval.**

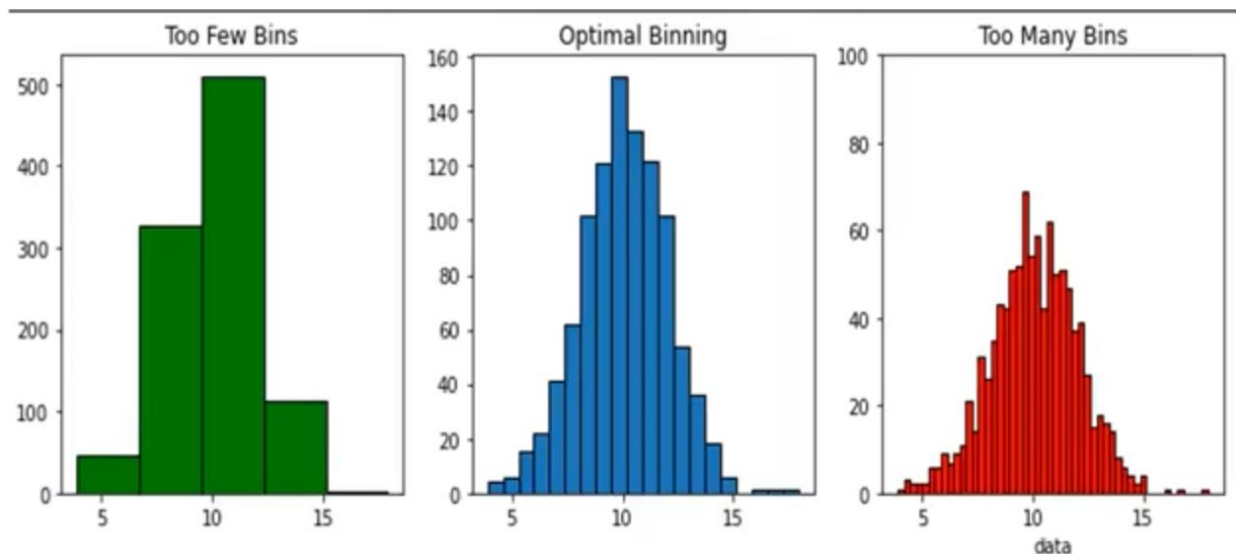
Histograms offer valuable insights into data distribution, outliers, skewness and variability.

They visually **depict the shape of the data**, whether it's symmetric skewed or bimodal.

Skewness can be assessed by examining the histogram's shape.

Histograms also showcase data variability, allowing you to **observe concentrations, gaps and clusters that reveal patterns or subgroups.**

Apart from issues with scale and inadequate labeling, be careful while choosing the bins to create a histogram. We have generated a random data set and created three histograms, with three different binning options. Too few bins, 5 represented in green color, an optimal number of bins, 20 represented in blue and too many bins 50, the yellow one.



By comparing these histograms, you can observe how **the choice of binning affects the representation of the data distribution** and results in a misleading representation of the data.

Selecting **too few or too many bins**, can oversimplify or overcomplicate the distribution, respectively.

In this video, you learned that there are various types of plots commonly used in data visualization.

- **Line plots** capture trends and changes over time, allowing us to see patterns and fluctuations.
- **Bar plots** compare categories or groups, providing a visual comparison of their values.
- **Scatter plots** explore relationships between variables, helping us identify correlations or trends.
- **Box plots** display the distribution of data, showcasing the median, quartiles and outliers.
- **Histograms** illustrate the distribution of data within specific intervals, allowing us to understand its shape and concentration.

Plot Libraries

After watching this video, you'll be able to

- **Explore** some popular plot **libraries** for data visualization.
- Identify the **features** of plot libraries in Python.

Data visualization is a vital tool for gaining insights and effectively communicating complex information.

Popular plot libraries

Plot libraries in Python:

- Matplotlib
- Pandas
- Seaborn
- Folium
- Plotly
- PyWaffle

Each plot library has its own strengths and use cases.

By harnessing the power of these plot libraries, you can unlock insights from your data and effectively communicate your findings.

Matplotlib.

It's a general-purpose plotting library that provides a wide variety of plots and customization options.



- General purpose plotting library
- Integrates well with libraries and frameworks

It integrates well with other libraries and frameworks such as NumPy, Pandas, Seaborn, and Plotly.

This interoperability allows you to combine the strengths of different libraries and create sophisticated and interactive visualizations.

Features.

- Line plots, scatter plots, bar charts, histograms, pie charts, box plots, and heat maps.
- Customize colors, line styles, marker styles, access labels, titles, legends.

Matplotlib's versatility, customization options, integration capabilities, and community support make it a powerful and reliable choice for **data visualization tasks in Python**.

Pandas.

Users primarily employ it for data manipulation, but it also offers **plotting capabilities, built on top of Matplotlib**, which means you can leverage the power and versatility of Matplotlib by combining it with Pandas data manipulation capabilities.

Features.

- It integrates seamlessly with Pandas data structures, making it easy to create line plots, scatter plots, bar charts, histograms, and pie charts **directly from your data frames**.

Pandas visualization capabilities are well suited for **exploratory data analysis**.

Seaborn.

Like Pandas, Seaborn is also **built upon the functionality of Matplotlib**.

It's a great choice for specialized **statistical visualizations**.

It provides stylish and specialized plot types such as categorical plots, count plots, heat maps, violin plots, scatter plots, bar plots, and many more.

Features.

- Various color **palettes and styles** to customize and enhance the aesthetics of the plot.
- Combine multiple plots into **grid layouts**, which can be useful for comparing multiple variables or subgroups.
- Plot data directly from Pandas data frames or series.

Seaborn's combination of statistical visualizations default, aesthetics integration with Pandas ease of use, and built in themes make it a popular choice among **data scientists and analysts for exploring and communicating insights** from data.

Folium.

When it comes to **geospatial data visualization**, Folium is an excellent library.

It allows you to create interactive and customizable **maps**.

Whether it's choropleth maps, point maps or heat maps, folium provides the tools to visually represent your geospatial data.

Feature.

- Seamlessly integrates with Pandas and NumPy.

It's a popular choice for geospatial data visualization and analysis in Python.

Plotly

If you want to present your data in an **interactive way**, plotly is a great choice to explore.

It offers highly interactive plots and dashboards.

With Plotly, you can create line plots, scatter plots, bar charts, pie charts, 3D plots, and Choropleth maps, to name a few.

Features.

Its Plotly dash framework allows you to build interactive dashboards with rich visualizations and controls.

Since Plotly is **web based**, it enables the rendering and viewing of plots in web browsers.

This makes it convenient for **sharing visualizations online**, embedding them in web applications or dashboards, and collaborating with others.

PyWaffle.

If you want to **visualize categorical data using Waffle** charts, PyWaffle is a simple yet effective library.

With PyWaffle, you can create waffle charts, square pie charts, donut charts, and many more types of plots by providing a unique way to represent proportions.

In this video, you learned that

- **Matplotlib** is a plotting library that offers a wide range of plotting capabilities.
- **Pandas** is a plotting library that provides integrated plotting functionalities for data analysis.
- **Seaborn** is a specialized library for **statistical visualizations**, offering attractive default aesthetics and color palettes.
- **Foleyum** is a Python library that allows you to create interactive and customizable maps.
- **Plotly** is an **interactive** and dynamic library for data visualization that supports a wide range of plot types and interactive features.
- **PyWaffle** enables you to visualize proportional representation using squares or rectangles.

Introduction to Matplotlib

After watching this video, you'll be able to explain

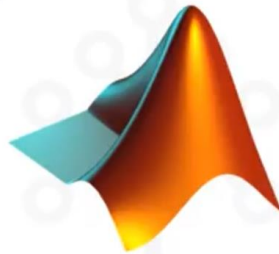
- What Matplotlib is and why it was created.
- Describe the uses for Matplotlib.

Matplotlib is one of the most widely used data visualization libraries in Python.

Matplotlib - history



John Hunter (1968 – 2012)



Matlab-based version of
a proprietary software



EEG/ECOG Visualization Tool

It was created by [John Hunter](#), who was an American neurobiologist.

[John Hunter](#) was part of a research team analyzing electro CTO cartography, [EECoG signals](#), and use proprietary software for this task.

However, the team had only one license and was taking turns using it.

To overcome this limitation, John Hunter set out to replace the proprietary software with a Matlab based version that could be utilized by him and his teammates and extended by multiple investigators.

As a result, Matplotlib was initially developed as an EEG and ECoG visualization tool.

Just like Matlab, Matplotlib was equipped with a scripting interface for quick and easy generation of graphics represented by the plot.

Matplotlib architecture



As for Matplotlib's architecture, it's composed of **three main layers**.

- The backend layer.
- The artist layer, where much of the heavy lifting happens, is the appropriate programming paradigm when writing a web application server, a UI application, or a script to be shared with other developers.
- The scripting layer is the appropriate layer for everyday purposes. It's considered a lighter scripting interface to simplify common tasks and for quick and easy generation of graphics and plots.

The backend layer.

The backend layer has three built-in abstract interface classes.

- FigureCanvas defines and encompasses the area on which the figure is drawn.
- Renderer, an instance of the renderer class knows how to draw on figure canvas.
- The event handles user inputs such as keyboard strokes and mouse clicks.

Backend layer

Has three built-in abstract interface classes:

1. FigureCanvas: **matplotlib.backend_bases.FigureCanvas**
 - Encompasses the area onto which the figure is drawn
2. Renderer: **matplotlib.backend_bases.Renderer**
 - Knows how to draw on the FigureCanvas
3. Event: **matplotlib.backend_bases.Event**
 - Handles user inputs such as keyboard strokes and mouse clicks

The artist layer.

The artist is the object that knows how to take the renderer and put ink on the Canvas.

Everything you see in a Matplotlib figure is an artist instance. The title, the lines, the tick labels, the images, and so on, all correspond to an individual artist.

Artist layer

- Comprised of one main object – **Artist**:
 - Knows how to use the Renderer to draw on the canvas.
- Title, lines, tick labels, and images, all correspond to individual **Artist** instances.
- Two types of **Artist** objects:
 1. **Primitive**: Line2D, Rectangle, Circle, and Text.
 2. **Composite**: Axis, Tick, Axes, and Figure.
- Each *composite* artist may contain other *composite* artists as well as *primitive* artists.

There are two types of artist objects:

- **Primitive**: Line2D, Rectangle, Circle, and Text.
- **Composite**: Axis, Tick, Axes, and Figure.

The **top-level** Matplotlib object that contains and manages all the elements in each graphic is the **figure** artist.

The most important composite artist is the **axis** because it's where most of the Matplotlib API plotting methods are defined, including methods to create and manipulate the ticks, the axis lines, the grid, and the plot background.

Each composite artists may contain other composite artists as well as primitive artists. A figure artists can have an axis artist, a rectangle, and a text artist.

The Scripting layer.

The artist layer is syntactically heavy. Programmers work directly with the backend and artists layers as they offer greater convenience while integrating Matplotlib with application servers.

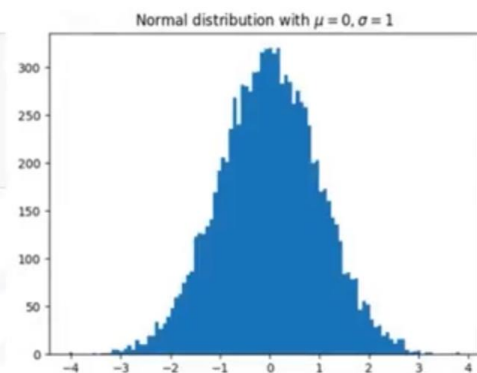
When it comes to the daily tasks of scientists involving data visualization or exploratory interactions, the scripting layer known as Pyplot works better.

Matplotlib scripting layer is the Matplotlib.Pyplot interface, which automatically defines a Canvas and a figure artist instance and connects them.

Let's generate a histogram of 10,000 random numbers with the scripting layer.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.randn(10000)
plt.hist(x, 100)
plt.title(r'Normal distribution with  $\mu=0$ ,  $\sigma=1$ ')
plt.savefig('matplotlib_histogram.png')
plt.show()
```



First we import the Pyplot interface and you can see all the methods associated with creating the histogram and other art objects and manipulating them.

Whether the hist method or showing the figure is part of the Pyplot interface.

Notice the use of Numpy's random module with random.randn for creating random floats.

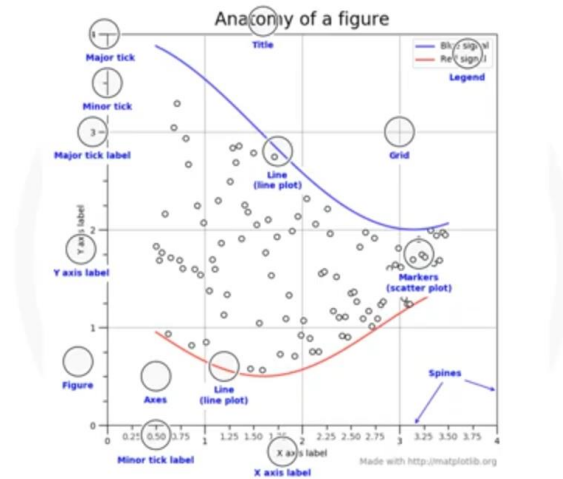
From the Pyplot hist method is called. hist creates a sequence of **rectangle artists** for each histogram bar and **adds them to the axes** container.

To the hist function, we have passed the variable X containing an array of 10,000 random numbers and 100, which means creating 100 bins. The result is a histogram.

It's simple to work in the scripting layer.

The [anatomy](#) of a plot refers to the different components and elements that make up a visual representation of data.

Components of a plot



SOURCE: <https://matplotlib.org/stable/gallery/showcase/anatomy.html>

This image provides a complete guide to understanding what a plot may include based on your requirements.

The main component is the window or Canvas containing the plot or subplots.

Matplotlib figure as a Canvas and axes represents an individual plot within a figure, you must first create two-axes parts to make two plots on one figure, the axis provides scales and tick marks for plotting the data.

The actual data being plotted is represented as points or markers on the plot like the axis.

A good plot must have a title to provide a summary or explanation of the plot.

Likewise, labels describe data being plotted on each axis.

You may like to include a legend to explain the meaning of different elements or data series and applaud a grid to help visually aligned data points and aid and reading values from the plot and annotations to provide supplemental information or explanations about specific data points are regions in the plot.

You can choose symbols for individual data point, colors, and styles.

This [link](#) will take you to a chapter written by the creators of Matplotlib. If you're interested in learning more about the history of Matplotlib and its architecture, then this is a recommended reading.

In this video, you learned that

Matplotlib is one of the most widely used data visualization libraries in Python.

Matplotlib was initially developed as an EEG, ECoG visualization tool.

Matplotlib architecture is composed of three main layers, the backend layer, the artist layer, and the scripting layer.

The anatomy of a plot refers to the different components and elements that make up a visual representation of data.

Basic Plotting with Matplotlib

- After watching this video, you'll be able to explore how to use
- Matplotlib to create plots by employing Jupyter notebook.

Create conventional visualization tools using the Plot function.

Matplotlib is a Well established data visualization library that can be integrated in different environments such as Python scripts, Python and IPython shells, web application servers and In Graphical User Interface toolkits. The Jupyter notebook is also one of them.

The Jupyter notebook is an open source web application that allows you to create and share documents that contain live code, visualizations and some explanatory text as well.

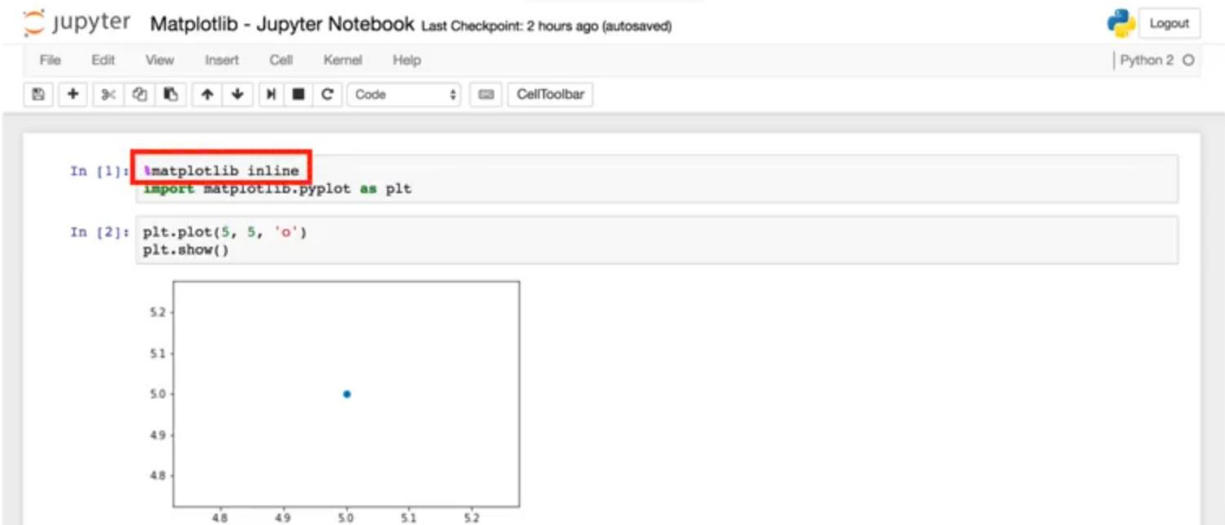
Jupyter has some specialized support for Matplotlib, so if you start a Jupyter notebook, all you must do is import Matplotlib and you're ready to go.

We will now learn how to create almost all the visualization tools using the Scripting interface, where you find out that you can literally create almost all of the conventional visualization tools, such as histograms, bar charts, boxplots and many others using one function only, the **Plot function**.

Let's start with an example.

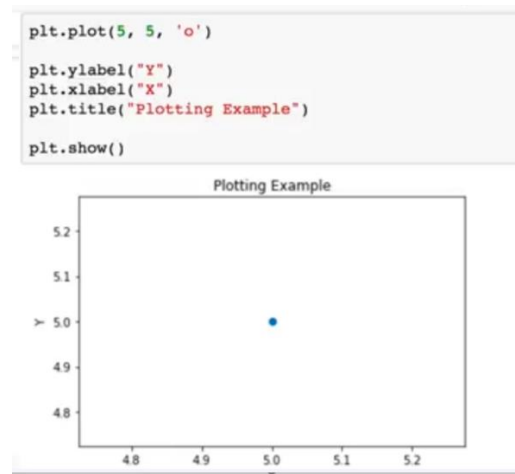
Let's first import the Scripting interface as plt and let's plot a circular mark at the position 5-5. Notice how the plot was **generated within the browser** and not in a separate window.

If the plot gets generated in the new popup window, then you can enforce generating plots into the browser using what's called the **magic function**, quote, `%matplotlib`, end quote, and you pass in inline as the back end.



Matplotlib has a number of different backends available:

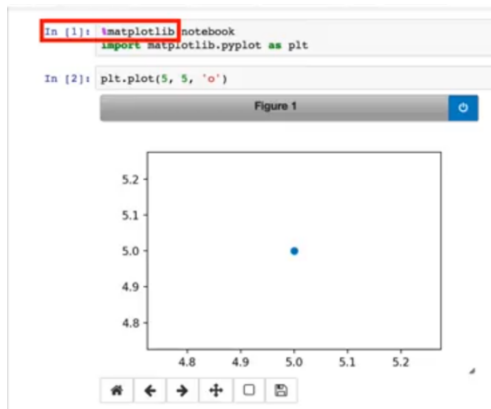
Backend inline.



One limitation of this backend is that you cannot modify a figure once it's rendered. So after rendering the figure, there is no way for us to add, for example, a figure title or labels to its axis. We will need to generate a new plot and add a title and the axis labels **before calling the show() function**.

Notebook backend.

With the notebook backend in place, if a plt function is called, it applies them to the active figure if it exists.



If a figure does not exist, it renders a new figure.

So when we call the `plt.plot` function to plot a circular mark at position 5-5, the backend checks if an active figure exists.

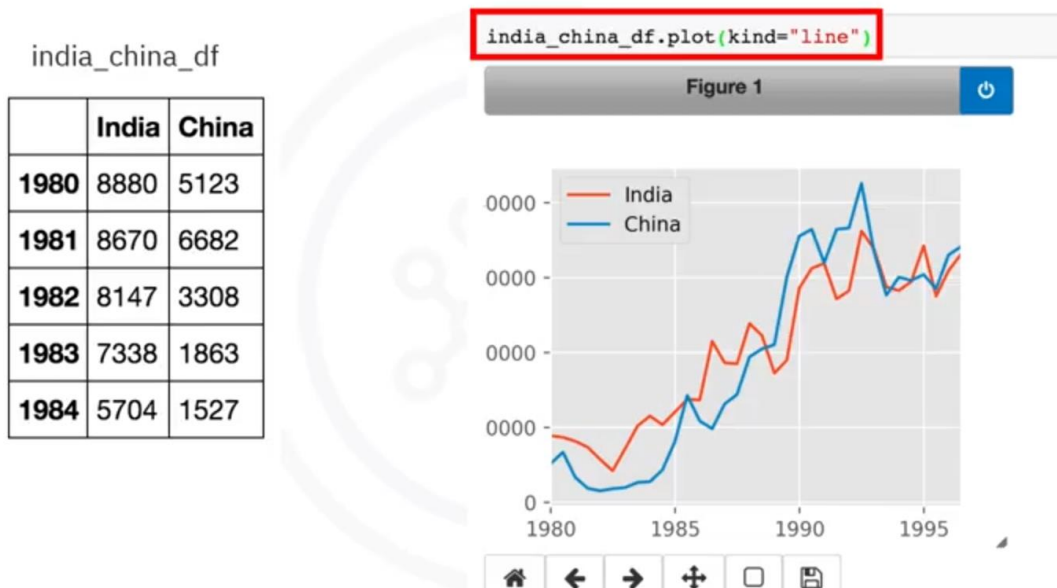
Since no active figure exists, it generates a figure and adds a circular mark to position 5-5. And what is beautiful about this backend is that now we can

easily add a title or labels to the axis after the plot was rendered without the need of regenerating the figure.

Matplotlib - pandas

Finally, another thing that is great about Matplotlib is that Pandas also has a **built-in implementation** of it. Therefore, plotting in Pandas is as simple as calling the `Plot` function on a given Pandas series or Pandas' data frame. So, say we have a data frame of the number of immigrants to Canada from India and China for the years 1980 to 1996.

Matplotlib – Pandas



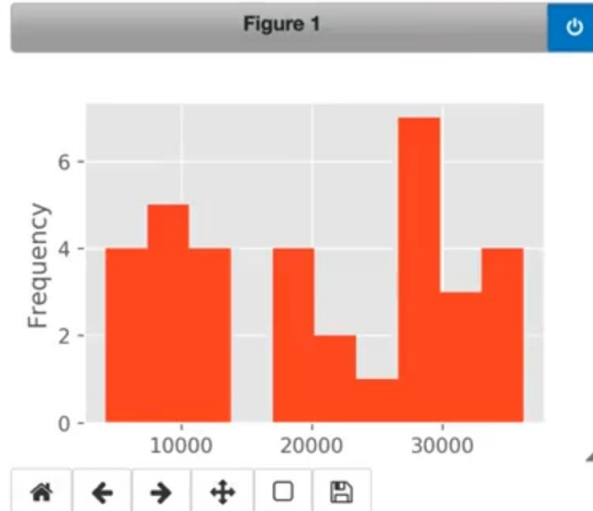
And we're interested in generating a **line plot** of this data. All we have to do is call the `plot` function on this data frame, which we have called `india_china_df` and pass in **`kind = line`**. And there you have it, a line plot of the data in the data frame.

Plotting a **histogram** of the data is not any different. So say we would like to plot a histogram of the India column in our data frame. All we have to do is call the plot function on that column and pass in kind as hist for histogram.

india_china_df

	India	China
1980	8880	5123
1981	8670	6682
1982	8147	3308
1983	7338	1863
1984	5704	1527

```
india_china_df["India"].plot(kind="hist")
```



And there you have it a histogram of the number of Indian immigrants to Canada from 1980 to 1996.

In this video, you learned that:

- Matplotlib is a well established data visualization library that can be integrated in different environments.
- Jupyter Notebook is an **open source web application** that allows you to create and share documents.
- Matplotlib has a number of **different backends** available.
- You can easily include the label and title to your plot with plt.

kind	Tipo de gráfico	Descripción breve
'line'	Gráfico de líneas	Predeterminado, útil para series temporales o datos continuos.
'bar'	Barras verticales	Comparación de valores por categoría.
'barh'	Barras horizontales	Igual que bar pero en horizontal.
'hist'	Histograma	Distribución de frecuencias de una variable numérica.

kind	Tipo de gráfico	Descripción breve
'box'	Diagrama de caja	Muestra la distribución estadística (mediana, cuartiles, outliers).
'kde' 'density'	Estimación de densidad kernel	Estimación suave de la distribución de probabilidad.
'area'	Gráfico de área	Similar al de líneas, pero con el área bajo la curva rellena.
'pie'	Gráfico de pastel	Muestra proporciones de un todo. Solo con Series o una columna.
'scatter'	Diagrama de dispersión	Requiere especificar x e y. Útil para ver relaciones entre dos variables.
'hexbin'	Hexagonal binning)	Densidad de puntos 2D, útil para grandes conjuntos de puntos. Requiere x e y.

Dataset on Immigration to Canada

What you will learn



Understand the dataset to be used in this course for data visualization



Import data with Pandas as a DataFrame in your program



Process data to make it suitable for plotting

We will use the Population Division of the United Nations compiled immigration data pertaining to 45 countries, where for each country, annual data corresponding to the total number of immigrants from all world countries are reported. In addition to other metadata pertaining to the immigrants' countries of origin. We'll focus on Canada and work primarily with the data set involving immigration to the Great White North.

Here is a snapshot of the UN data on immigration to Canada in the form of an Excel file. The first 20 rows contain textual data about the UN department and other information. Row 21 contains the labels of the columns.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	 United Nations Population Division Department of Economic and Social Affairs International Migration Flows to and from Selected Countries: The 2015 Revision POP/DB/MIG/Flow/Rev.2015 December 2015 - Copyright © 2015 by United Nations. All rights reserved Suggested citation: United Nations, Department of Economic and Social Affairs, Population Division (2015). International Migration Flows to and from Selected Countries: The 2015 Revision. (United Nations database,														
2															
3															
4															
5															
6															
7															
8															
9															
10															
11															
12															
13															
14															
15															
16															
17	Reporting country: Canada														
18	Criterion: Citizenship														
19															
20	Classification		Origin/Destination	Major area		Region		Development region							
21	Type	Coverage	OdName	AREA	AreaName	REG	RegName	DEV	DevName	1980	1981	1982	1983	1984	1985
22	Immigrants	Foreigners	Alghanistan	935	Asia	5501	Southern Asia	902	Developing regions	16	39	39	47	71	340
23	Immigrants	Foreigners	Albania	908	Europe	925	Southern Europe	901	Developed regions	1	0	0	0	0	0
24	Immigrants	Foreigners	Algeria	903	Africa	912	Northern Africa	902	Developing regions	80	67	71	69	63	44
25	Immigrants	Foreigners	American Samoa	909	Oceania	957	Polynesia	902	Developing regions	0	1	0	0	0	0
26	Immigrants	Foreigners	Andorra	908	Europe	925	Southern Europe	901	Developed regions	0	0	0	0	0	0

Following that, each row represents a country along with metadata about the country, such as the continent it resides in, what region it belongs to, and whether the region is developing or developed. Then we have the total number of immigrants from that country for the years 1980 all the way to 2013.

Read data into Pandas DataFrame

```
import numpy as np # useful for many scientific computing in Python
import pandas as pd # primary data structure library
from __future__ import print_function # adds compatibility to python 2
```

```
#install openpyxl
!pip install openpyxl
print('openpyxl installed')
```

```
df_can = pd.read_excel(
    'https://ibm.box.com/shared/static/lw190pt9zpy5bd1ptyg2aw15awomz9pu.xlsx',
    sheetname="Canada by Citizenship",
    skiprows=range(20),
    skip_footer = 2)
```

Throughout this course, we will be using pandas for any analysis of the data before creating visualization tools. So in order to start creating different types of plots of the data, whether

for exploratory analysis or for presentation, we will need to **import the data into a pandas DataFrame**.

To do that, we will need to import the pandas library, as well as the openpyxl library, which is required to extract data from Excel spreadsheets files. Then we will call the pandas function, read excel. To read the data into a pandas DataFrame, notice how we're **skipping the first 20 rows** to read only the data corresponding to each country.

If you want to confirm that you have imported your data correctly, you can always use the head function to display the first five rows in the DataFrame.

Now, let's process the data frame so that **the country name becomes the index** of each row. This should **make querying specific countries easier**. Also, let's add an extra column which represents the total immigration for each country from 1980 to 2013. So, for Afghanistan it's 58,639, and for Albania it's 15,699, and so on.

Now let's name our DataFrame df_canada.

Recap

In this video, you learned that:

- The population division of the United Nations compiled immigration data pertaining to 45 countries.
- The UN data on immigration to Canada shows data related to the number of people migrated.
- In order to start creating different types of plots of the data, you will need to import the data into a Pandas DataFrame.

Line Plots

What you will learn



Describe line plot
and its function



Determine when to
use a line plot



Create a line plot
from data in the
dataset

What is a line plot?

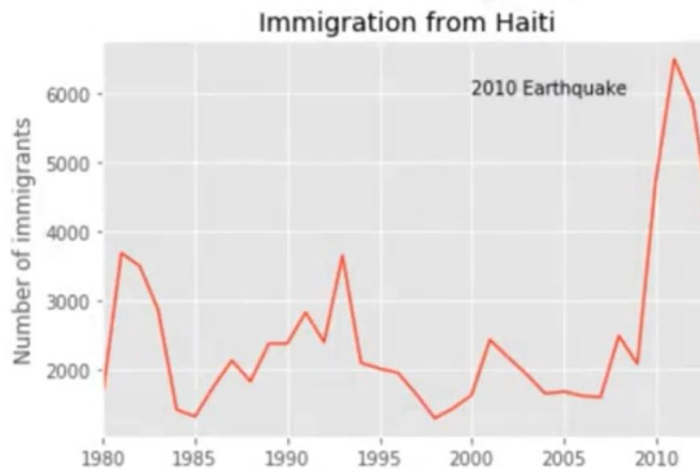
It's a plot that displays information as a series of data points connected by straight lines. It is one of the most basic types of charts and is common in many fields, not just data science.

when to use a line plot?

- For **visualizing trends and changes over time**, making them a popular choice for time-series data, such as changes in stock prices, website traffic, or temperature fluctuations.
- To **show relationships between two variables**.
- To **compare multiple data series** on one chart.
- **highlight sudden changes** or anomalies in data.

Line plots

Spike of immigration from Haiti to Canada in 2010 due to earthquake



As an example, from our dataset, we can generate a line plot to see the trend of immigrants from Haiti to Canada. Based on this line plot, we see that there is a spike of immigration from Haiti to Canada in 2010.

We can then research for justifications of obvious anomalies or changes from a quick Google search for major events in Haiti in 2010 one would easily learn about the tragic earthquake that took place in 2010. Therefore, this influx of immigration to Canada was mainly due to that tragic earthquake.

Now, how can we generate this line plot, as we briefly mentioned in an earlier video with matplotlib, all we must do is call the plot function on the pandas data frame or series containing the data of interest.

Dataset - recap

	Type	Coverage	OdName	AREA	AreaName	REG	RegName	DEV	DevName	1980	...	2004	2005	2006	2007	2008	2009	2010	2011
0	Immigrants	Foreigners	Afghanistan	935	Asia	5501	Southern Asia	902	Developing regions	16	...	2978	3436	3009	2652	2111	1746	1758	2203
1	Immigrants	Foreigners	Albania	908	Europe	925	Southern Europe	901	Developed regions	1	...	1450	1223	856	702	560	716	561	539
2	Immigrants	Foreigners	Algeria	903	Africa	912	Northern Africa	902	Developing regions	80	...	3616	3626	4807	3623	4005	5393	4752	4325
3	Immigrants	Foreigners	American Samoa	909	Oceania	957	Polynesia	902	Developing regions	0	...	0	0	1	0	0	0	0	0
4	Immigrants	Foreigners	Andorra	908	Europe	925	Southern Europe	901	Developed regions	0	...	0	0	1	1	0	0	0	0

Now let's process the DataFrame so that the **country name becomes the index of each row**. This should make querying specific countries easier.

Dataset - processed

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3009	2652	2111	1746	1758	2203	2635	2004	58639
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	856	702	560	716	561	539	620	603	15699
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	4807	3623	4005	5393	4752	4325	3774	4331	69439
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	1	0	0	0	0	0	0	0	6
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	1	1	0	0	0	0	1	1	15

Also, let's **add** an extra **column** which represents the **total** immigration for each country, from 1980-2013.

Let's generate the line plot corresponding to immigration from Haiti.

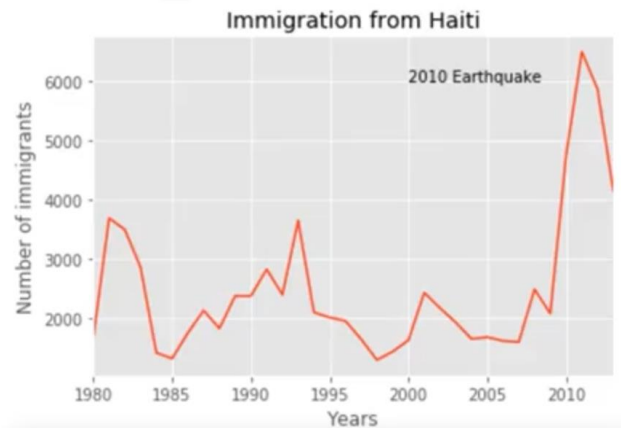
Creating line plots

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
years = list(map(str, range(1980, 2014)))

df_canada.loc['Haiti', years].plot(kind = 'line')
plt.title('Immigration from Haiti')
plt.ylabel('Number of immigrants')
plt.xlabel('Years')

plt.show()
```



Recap

In this video, you learned that:

- A line plot is a plot in the form of a series of data points connected by straight line segments.
- Line plot is one of the most basic type of chart and is common in many fields.
- You can generate a line plot by assigning "line" to 'Kind' parameter in the plot() function.

Hands-on Lab: Exploring and Pre-processing a Dataset using Pandas

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...	2005	2006	2007	2008	2009	2010	2
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	3436	3009	2652	2111	1746	1758	2
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1223	856	702	560	716	561	

Poner como índice del DataFrame los valores de una columna:

```
df_can.set_index('Country', inplace=True)
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3009	2652	2111	1746	1758	2203	2635	2004	58639
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	856	702	560	716	561	539	620	603	15699
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	4807	3623	4005	5393	4752	4325	3774	4331	69439

Remover el nombre del índice: `df_can.index.name = None`

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3009	2652	2111	1746	1758	2203	2635	2004	58639
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	856	702	560	716	561	539	620	603	15699
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	4807	3623	4005	5393	4752	4325	3774	4331	69439

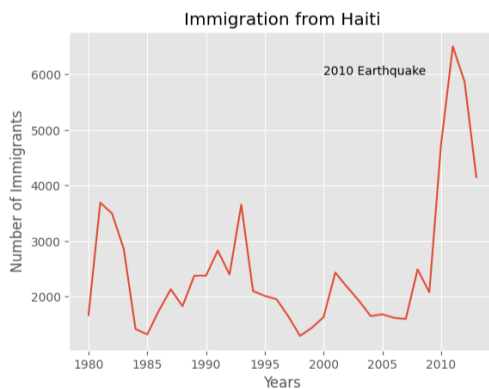
```
df_can[df_can.index == 'Japan']
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Japan	Asia	Eastern Asia	Developed regions	701	756	598	309	246	198	248	...	1067	1212	1250	1284	1194	1168	1265	1214	982	27707

convert the column names into strings: '1980' to '2013': `df_can.columns = list(map(str, df_can.columns))`

Hands-on Lab: Introduction to Matplotlib and Line Plots

Interesante comentario para las coordenadas de las etiquetas



With just a few lines of code, you were able to quickly identify and visualize the spike in immigration!

Quick note on x and y values in `plt.text(x, y, label)`:

Since the x-axis (years) is type 'integer', we specified x as a year. The y axis (number of immigrants) is type 'integer', so we can just specify the value y = 6000.

```
plt.text(2000, 6000, '2010 Earthquake') # years stored as type int
```

If the years were stored as type 'string', we would need to specify x as the index position of the year. Eg 20th index is year 2000 since it is the 20th year with a base year of 1980.

```
plt.text(20, 6000, '2010 Earthquake') # years stored as type int
```

We will cover advanced annotation methods in later modules.

Tambien la necesidad de transponer el Dataframe para hacer un grafico de lineas

That doesn't look right..

Recall that *pandas* plots the indices on the x-axis and the columns as individual lines on the y-axis. Since `df_CI` is a dataframe with the `country` as the index and `years` as the columns, we must first transpose the dataframe using `transpose()` method to swap the row and columns.

```
[36]: df_CI = df_can.loc[['India', 'China'], years]
df_CI
```

```
[36]:
```

	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	...	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013
India	8880	8670	8147	7338	5704	4211	7150	10189	11522	10343	...	28235	36210	33848	28742	28261	29456	34235	27509	30933	33087
China	5123	6682	3308	1863	1527	1816	1960	2643	2758	4323	...	36619	42584	33518	27642	30037	29622	30391	28502	33024	34129

2 rows × 34 columns

```
[37]: df_CI = df_CI.transpose()
df_CI.head()
```

```
[37]:
```

	India	China
1980	8880	5123
1981	8670	6682
1982	8147	3308
1983	7338	1863
1984	5704	1527

pandas will automatically graph the two countries on the same graph. Go ahead and plot the new transposed dataframe. Make sure to add a title to the plot and label the axes.

Practice Quiz: Introduction to Data Visualization

Module 1 Summary: Introduction to Data Visualization Tools

- Data visualization is the process of presenting data in a visual format, such as charts, graphs, and maps, to help people understand and analyze data easily.
- Data visualization has diverse use cases, such as in business, science, healthcare, and finance.
- It is important to follow best practices, such as **selecting appropriate visualizations for the data being presented**, choosing colors and fonts that are easy to read and interpret, and minimizing clutter.
- There are various types of plots commonly used in data visualization.
- Line plots capture trends and changes over time, allowing us to see patterns and fluctuations.
- Bar plots compare categories or groups, providing a visual comparison of their values.
- Scatter plots explore relationships between variables, helping us identify correlations or trends.
- Box plots display the distribution of data, showcasing the median, quartiles, and outliers.
- Histograms illustrate the distribution of data within specific intervals, allowing us to understand its shape and concentration.

- Matplotlib is a plotting library that offers a wide range of plotting capabilities.
- Pandas is a plotting library that provides Integrated plotting functionalities for data analysis.
- Seaborn is a specialized library for **statistical visualizations**, offering attractive default aesthetics and color palettes.
- Folium is a Python library that allows you to create interactive and customizable **maps**.
- Plotly is an **interactive** and dynamic library for data visualization that supports a wide range of plot types and interactive features.
- PyWaffle enables you to visualize **proportional representation using squares** or rectangles.
- Matplotlib is one of the most widely used data visualization libraries in Python.
- Matplotlib was initially developed as an EEG/ECOG visualization tool.
- Matplotlib's architecture is composed of three main layers: Backend layer, Artist layer, and the Scripting layer.
- The anatomy of a plot refers to the different components and elements that make up a visual representation of data.
- Matplotlib is a well-established data visualization library that can be integrated in different environments.
- Jupyter Notebook is an open-source web application that allows you to **create and share documents**.
- Matplotlib has several different **backends available**.
- You can easily include the label and title to your plot with plt.
- To start creating different types of plots of the data, you will need to import the data into a Pandas DataFrame.
- A line plot is a plot in the form of a series of data points connected by straight line segments.
- Line plot is one of the most basic types of chart and is common in many fields.
- You can generate a line plot by assigning "line" to 'Kind' parameter in the plot() function.

Module 1 Cheat Sheet

Module 1 Graded Quiz: Introduction to Data Visualization Tools

Module 2

Title: Basic and Specialized Visualization Tools

Description

Visualization tools play a crucial role in data analysis and communication. These are essential for extracting insights and presenting information in a concise manner to both technical and non-technical audiences. In this module, you will create a diverse range of plots using Matplotlib, the data visualization library. Throughout this module, you will learn about area plots, histograms, bar charts, pie charts, box plots, and scatter plots. You will also explore the process of creating these visualization tools using Matplotlib.

Objectives

By the end of this week, you will be able to:

- Explore an area plot with an illustration and create it using Matplotlib
- Define a histogram with an illustration and create it using Matplotlib
- Describe a bar chart with an illustration and create it using Matplotlib
- Discover a pie chart with an illustration and create it using Matplotlib
- Describe a box plot with an illustration and create it using Matplotlib
- Discover a scatter plot with an illustration and create it using Matplotlib

Activities

Lesson 1: Basic Visualization Tools

Area Plots

What you will learn



Describe what is an area plot



Explain how to create an area plot using Matplotlib

What is an area plot?

An area plot, also known as an area chart or graph, displays the magnitude and proportion of multiple variables over a continuous axis, typically representing time or another ordered dimension. It's like a line plot, but with the area below the line filled with color to **emphasize the cumulative magnitude of the variables**. This kind of graph is commonly used when trying to **compare two or more quantities**.

Dataset: Processed

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3009	2652	2111	1746	1758	2203	2635	2004	58639
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	856	702	560	716	561	539	620	603	15699
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	4807	3623	4005	5393	4752	4325	3774	4331	69439
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	1	0	0	0	0	0	0	0	6
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	1	1	0	0	0	0	1	1	15

Let's try to generate area plots for the countries with the highest immigration to Canada, sorting the dataset by Total column.

Generating area plots

```
df_canada.sort_values(['Total'], ascending = False, axis = 0, inplace = True)
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	4211	7150	...	36210	33848	28742	28261	29456	34235	27509	30933	33087	691904
China	Asia	Eastern Asia	Developing regions	5123	6682	3308	1863	1527	1816	1960	...	42584	33518	27642	30037	29622	30391	28502	33024	34129	659962
United Kingdom of Great Britain and Northern Ireland	Europe	Northern Europe	Developed regions	22045	24796	20620	10015	10170	9564	9470	...	7258	7140	8216	8979	8876	8724	6204	6195	5827	551500
Philippines	Asia	South-Eastern Asia	Developing regions	6051	5921	5249	4562	3801	3150	4166	...	18139	18400	19837	24887	28573	38617	36765	34315	29544	511391
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	514	691	...	14314	13127	10124	8994	7217	6811	7468	11227	12603	241600

It turns out that India followed by China, then the United Kingdom, the Philippines, and Pakistan, are the top five countries with the highest immigration to Canada.

Can we now go ahead and generate the area plots using the first five rows of this DataFrame? Not quite yet.

First, we need to create a new DataFrame of these five countries only and **exclude** the **total** column. More importantly, to generate the area plots for these countries, we should **plot the years on the horizontal axis and the annual immigration on the vertical axis**.

Country	India	China	United Kingdom of Great Britain and Northern Ireland	Philippines	Pakistan
1980	8880	5123	22045	6051	978
1981	8670	6682	24796	5921	972
1982	8147	3308	20620	5249	1201
1983	7338	1863	10015	4562	900
1984	5704	1527	10170	3801	668

Note that Matplotlib plots the indices of a DataFrame on the horizontal axis and with the DataFrame as shown, Matplotlib plots the countries on the horizontal axis.

To fix this, we need to take the **transpose** of the DataFrame.

```
years = list(map(str, range(1980, 2014)))

df_canada.sort_values(['Total'], ascending = False, axis = 0, inplace = True)

df_top5 = df_canada.head()
df_top5 = df_top5[years].transpose()
```

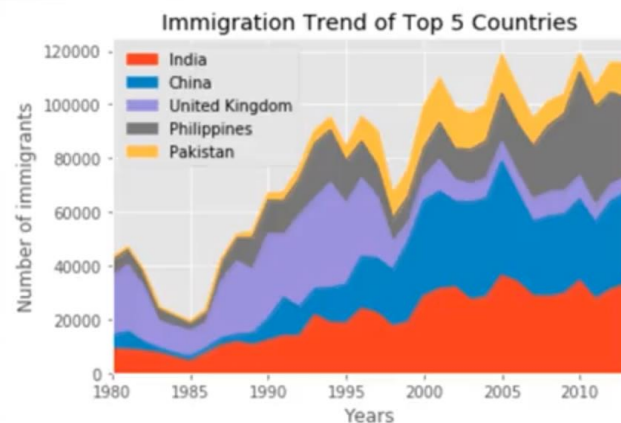
Now we can go ahead and use the plot function on dataframe, df_top five to generate the area plots.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
df_top5.plot(kind='area')

plt.title('Immigration trend of top 5 countries')
plt.ylabel('Number of immigrants')
plt.xlabel('Years')

plt.show()
```



Area plots are particularly effective at depicting data with a **cumulative nature**, such as tracking stock market performance, visualizing population demographics, or displaying the distribution of resources across various sectors.

Recap

In this video, you learned that:

- An area plot depicts cumulated totals using numbers or percentages over time.
- The process of creating an area plot involves importing Matplotlib and calling the plot function on the dataframe with 'kind' parameter assigned as 'area.'
- Area plots provide a visually appealing and intuitive way to showcase the relationship and proportion of multiple variables in a single chart.

What you will learn



Define a histogram with the help of an illustration



Explore the process of creating a histogram using Matplotlib

A histogram is a way of representing the **frequency distribution** of a numeric data set.

The way it works is that it partitions the spread of the numeric data into bins, assigns each data point in the data set to a bin, and then counts the number of data points assigned to each bin.

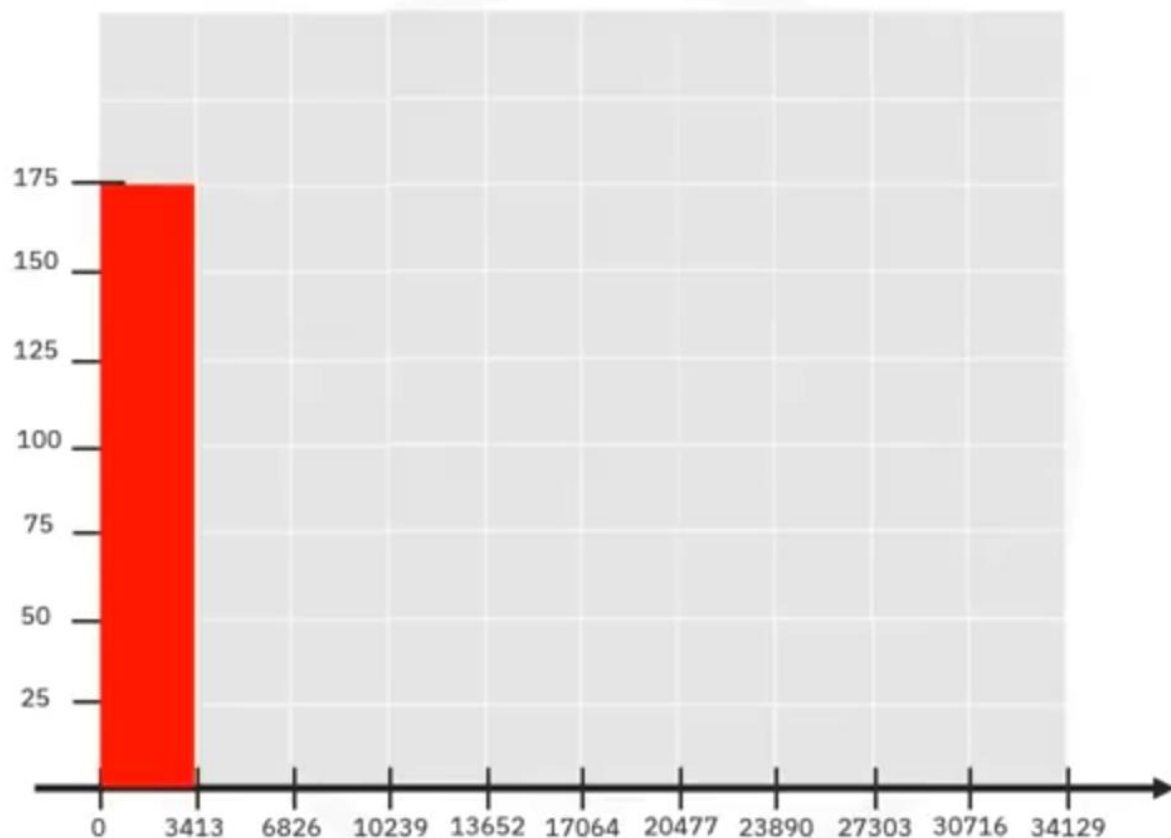
So, the **vertical axis** is essentially the frequency or the **number of data points in each bin**.

For example, let's say the range of the numeric values in the data set is from zero to 34,129.

Now, **the first step** in creating a histogram is **partitioning the horizontal axis** in, say, 10 bins of equal width.

Then we construct the histogram by counting how many data points have a value that is between the limits of the first bin, the second bin, the third bin, and so on.

Say, the number of data points that have a value between 0 and 3,413 is 175, then we draw a **bar of that height for this bin**, we repeat the same step for all the other bins. And if no data points fall into a bin, then that bin would have a bar of height zero.



So how do we create a histogram using Matplotlib?

Say we want to visualize the distribution of immigrants to Canada in the year 2013.

Generating a histogram

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3009	2652	2111	1746	1758	2203	2635	2004	58639
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	856	702	560	716	561	539	620	603	15699
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	4807	3623	4005	5393	4752	4325	3774	4331	69439
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	1	0	0	0	0	0	0	0	6
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	1	1	0	0	0	0	1	1	15

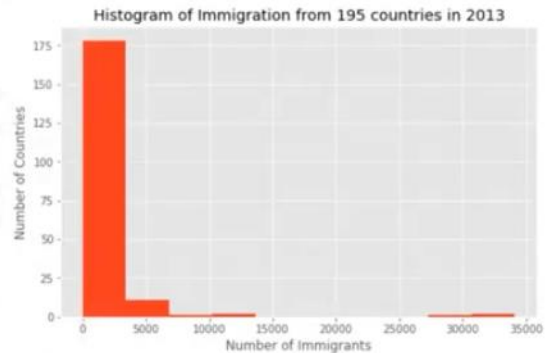
The simplest way to do that is to generate a histogram of the data in column 2013. a histogram that depicts the distribution of immigration to Canada in 2013.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
df_canada['2013'].plot(kind='hist')

plt.title('Histogram of Immigration from 195 countries in 2013')
plt.ylabel('Number of Countries')
plt.xlabel('Number of Immigrants')

plt.show()
```



But notice how the **bins are not aligned with the tick marks** on the horizontal axis, this can make the histogram hard to read.

So, let's try to **fix** this to **make** our **histogram more effective** borrowing the histogram function from the NumPy Library.

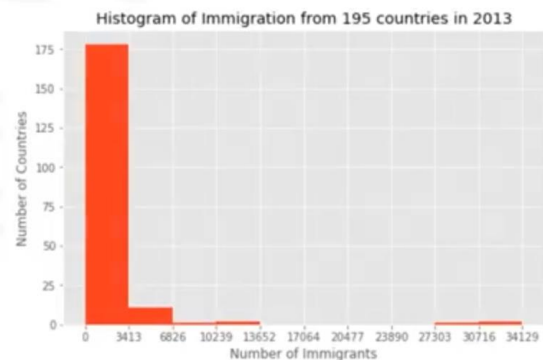
```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
```

```
count, bin_edges = np.histogram(df_canada['2013'])

df_canada['2013'].plot(kind='hist', xticks = bin_edges)

plt.title('Histogram of Immigration from 195 countries in 2013')
plt.ylabel('Number of Countries')
plt.xlabel('Number of Immigrants')

plt.show()
```



We also import the NumPy library, then we call the NumPy histogram function on the data in column 2013. This function is going to partition the spread of the data in column 2013 in ten bins of equal width, where **ten is the default number of bins**.

It also computes the number of data points that fall in each bin and then return this frequency of each bin which we are calling 'count' here, and the bin edges which we will call 'bin_edges'.

We then pass these bin edges as an additional parameter in our plot function to generate the histogram, and there you go. **A precisely generated histogram with the bin edges and tick marks clearly aligned on the horizontal axis.**

Recap

In this video, you learned that:

- A histogram is a way of representing the frequency distribution of a numeric dataset.
- To generate a histogram on Matplotlib, you import Matplotlib as `mpl` and its scripting interface as `plt`.
- You can call the `plot` function on the dataframe with `kind` parameter assigned as `'hist.'`
- You can use the Numpy library to create bins for the histogram representation.

Bar Charts

What you will learn

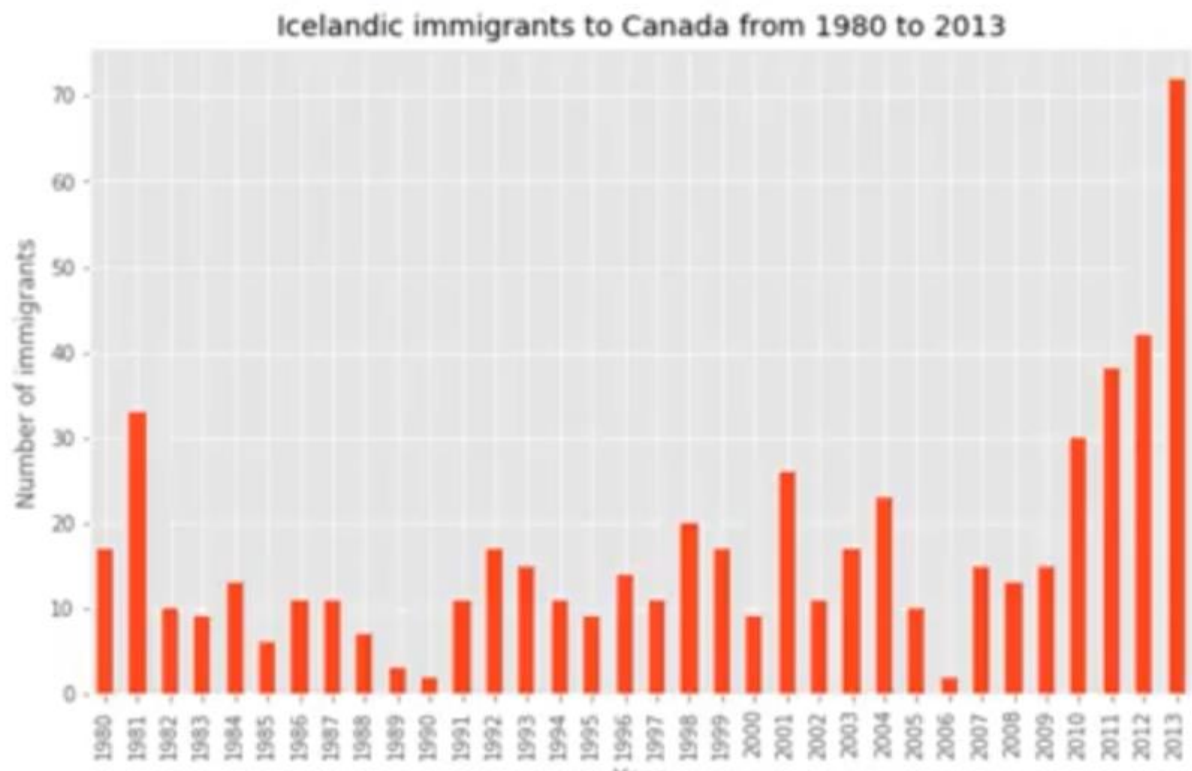


Describe a bar chart with the help of an illustration



Explore the process of creating a bar chart using Matplotlib

A bar chart is a popular visualization tool. Unlike a histogram, a bar chart, also known as a bar graph, is a type of plot where **the length of each bar is proportional to the value of the item that it represents**. It's commonly used to compare the values of a variable at a given point in time.



For example, say we want to visualize in a discrete fashion, how immigration from Iceland to Canada looked 1980-2013.

How do we do that with Matplotlib?

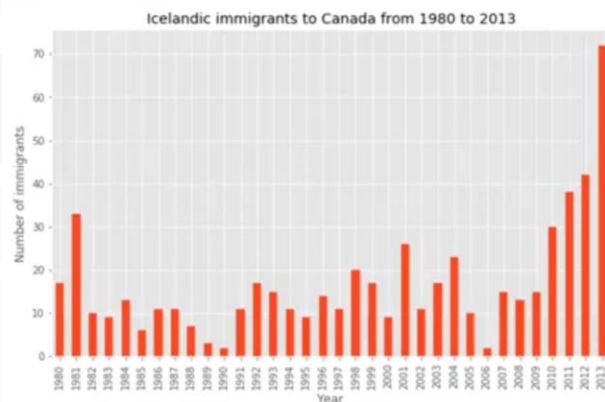
```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
Years = list (map(str, range (1980, 2014)))
df_iceland = df_canada.loc[ 'Iceland',
years]
```

```
df_iceland.plot(kind='bar')

plt.title('Icelandic immigrants to
Canada from 1980 to 2013')
plt.xlabel('Year')
plt.ylabel('Number of immigrants')

plt.show()
```

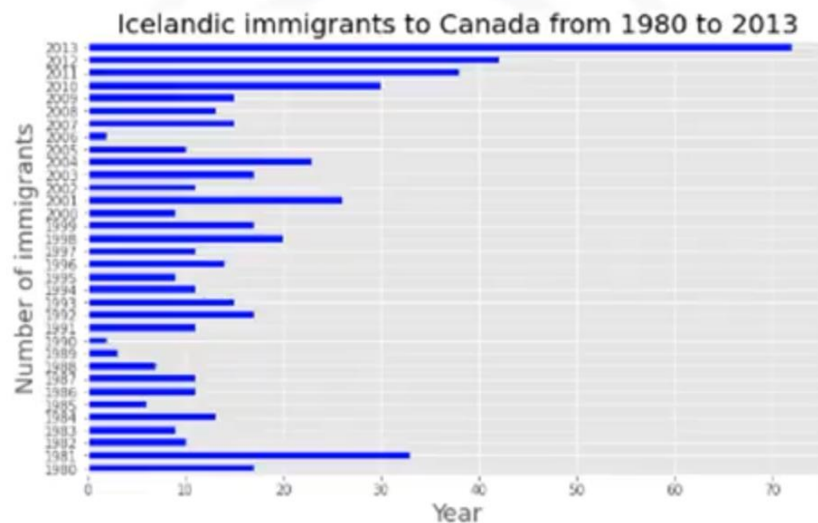


By examining the bar chart, we noticed that immigration to Canada from Iceland has seen an increasing trend since 2010.

You can also create a bar chart with horizontal bars by assigning bar to the kind parameter of the plot function.

Horizontal bar chart

```
df_iceland.plot(kind='barh', color = 'red')
```

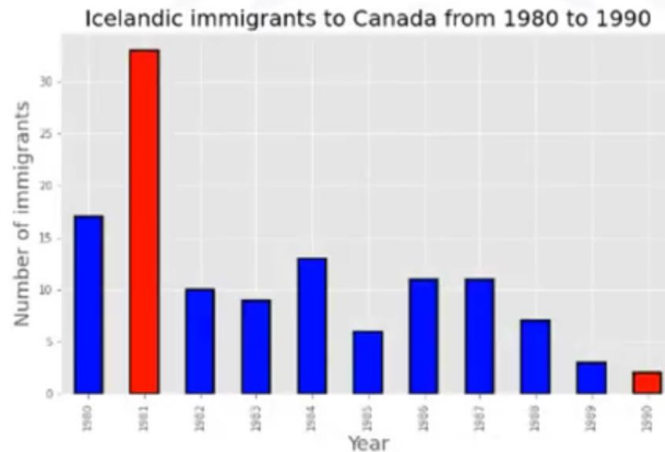


Note the use of the color parameter as you can change the color of the bar with this.

Let's suppose you want to highlight the years with highest and lowest number of Icelandic immigrants to Canada between the year 1980 to 1990. You can pass a list of colors to the color parameter accordingly. Here we have highlighted the bars for the years 1981 and 1990 with the color red. By assigning the color of your choice to the edge color parameter, you can change the borderline color of each bar.

Highlighted bar and edge color

```
c=['blue','red',"blue",'blue','blue','blue','blue','blue','blue','blue','red']  
df_Iceland.plot(kind = 'bar', color = c, edgecolor = 'black')
```



Recap

In this video, you learned that:

- A bar chart is a type of plot where the length of each bar is proportional to the value of the item that it represents.
- You can create a bar chart using Matplotlib representing the total immigration from Iceland to Canada.

Hands-on Lab: Area Plots, Histograms, and Bar Charts

Area plots

Area plots are stacked by default. And to produce a stacked area plot, each column must be either all positive or all negative values (any NaN, i.e. not a number, values will default to 0). **To produce an unstacked plot, set parameter stacked to value False.**

The unstacked plot has a **default transparency (alpha value) at 0.5**. We can modify this value by passing in the alpha parameter.

Two types of plotting

Option 1: Scripting layer (procedural method) - using matplotlib.pyplot as 'plt'

You can use plt i.e. matplotlib.pyplot and add more elements by calling different methods procedurally; for example, plt.title(...) to add title or plt.xlabel(...) to add label to the x-axis.

```
# Option 1: This is what we have been using so far
df_top5.plot(kind='area', alpha=0.35, figsize=(20, 10))
plt.title('Immigration trend of top 5 countries')
plt.ylabel('Number of immigrants')
plt.xlabel('Years')
```

Option 2: Artist layer (Object oriented method) - using an Axes instance from Matplotlib (preferred)

You can use an **Axes** instance of your current plot and store it in a variable (eg. **ax**). You can add more elements by calling methods with a little change in syntax (by adding "set_" to the previous methods). For example, use **ax.set_title()** instead of **plt.title()** to add title, or **ax.set_xlabel()** instead of **plt.xlabel()** to add label to the x-axis.

This option sometimes is more transparent and flexible to use for advanced plots (in particular when having multiple plots, as you will see later).

In this course, we will stick to the **scripting layer**, except for some advanced visualizations where we will need to use the **artist layer** to manipulate advanced aspects of the plots.

```
# option 2: preferred option with more flexibility
ax = df_top5.plot(kind='area', alpha=0.35, figsize=(20, 10))

ax.set_title('Immigration Trend of Top 5 Countries')
ax.set_ylabel('Number of Immigrants')
ax.set_xlabel('Years')
```

Histograms

Para alinear las barras con las marcas del eje x usamos la función np.histogram que nos crea los cestos iguales (10, por defecto) en los que repartir los numeros.

```

count, bin_edges = np.histogram(df_t, 15)
xmin = bin_edges[0] - 10 # first bin value is 31.0, adding buffer of 10 for aesthetic purposes
xmax = bin_edges[-1] + 10 # Last bin value is 308.0, adding buffer of 10 for aesthetic purposes

# stacked Histogram
df_t.plot(kind='hist',
          figsize=(10, 6),
          bins=15,
          xticks=bin_edges,
          color=['coral', 'darkslateblue', 'mediumseagreen'],
          stacked=True,
          xlim=(xmin, xmax)
         )

plt.title('Histogram of Immigration from Denmark, Norway, and Sweden from 1980 - 2013')
plt.ylabel('Number of Years')
plt.xlabel('Number of Immigrants')

plt.show()

```

Bar Charts

Vemos las anotaciones

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.annotate.html

```

df_iceland.plot(kind='bar', figsize=(10, 6), rot=90)

plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.title('Icelandic Immigrants to Canada from 1980 to 2013')

# Annotate arrow
plt.annotate('', # s: str. will leave it blank for no text
             xy=(32, 70), # place head of the arrow at point (year 2012 , pop 70)
             xytext=(28, 20), # place base of the arrow at point (year 2008 , pop 20)
             xycoords='data', # will use the coordinate system of the object being annotated
             arrowprops=dict(arrowstyle='->', connectionstyle='arc3', color='blue', lw=2)
            )

# Annotate Text
plt.annotate('2008 - 2011 Financial Crisis', # text to display
             xy=(28, 30), # start the text at at point (year 2008 , pop 30)
             rotation=72.5, # based on trial and error to match the arrow
             va='bottom', # want the text to be vertically 'bottom' aligned
             ha='left', # want the text to be horizontally 'left' aligned.
            )

plt.show()

```

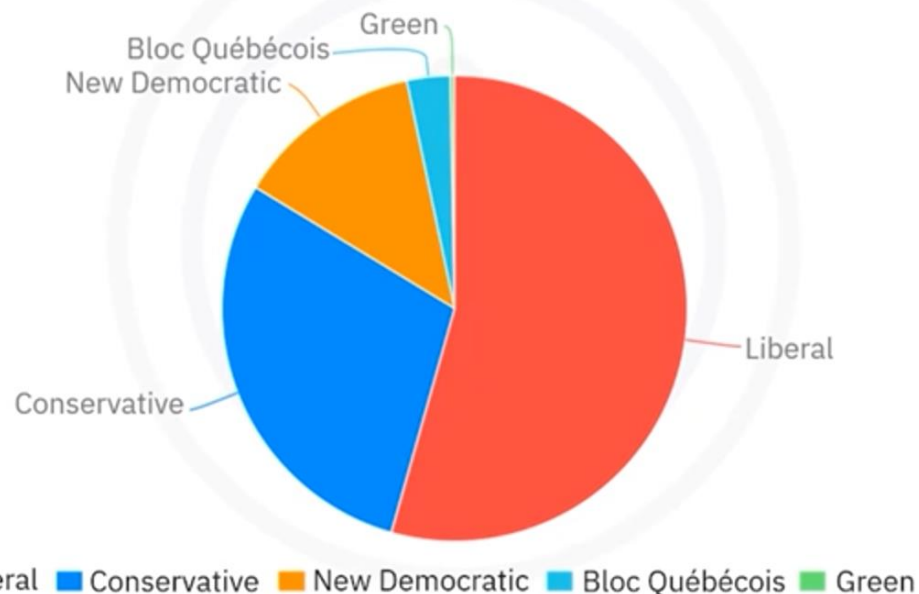
Practice Quiz: Basic Visualization Tools

Lesson 2: Specialized Visualization Tools

Pie Charts

It is a circular statistical graphic divided into segments to illustrate **numerical proportion**.

A circular statistical graphic, divided into segments



How to create a pie chart with Matplotlib.

```
df_continents = df_canada.groupby('Continent', axis = 0).sum()
```

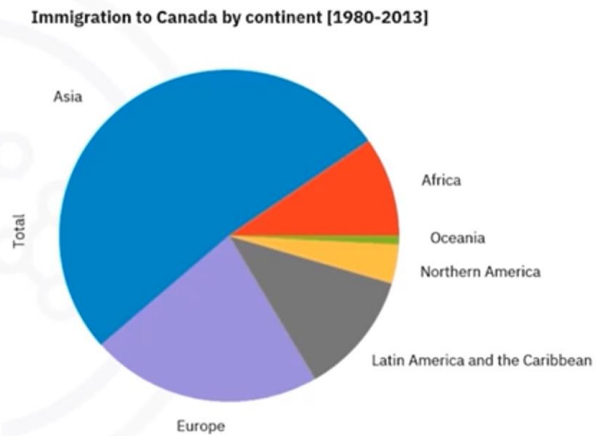
	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Continent																					
Africa	3951	4363	3819	2671	2639	2650	3782	7494	7552	9894	...	27523	29188	28284	29890	34534	40892	35441	38083	38543	618948
Asia	31025	34314	30214	24696	27274	23850	28739	43203	47454	60256	...	159253	149054	133459	139894	141434	163845	146894	152218	155075	3317794
Europe	39760	44802	42720	24638	22287	20844	24370	46698	54726	60893	...	35955	33053	33495	34692	35078	33425	26778	29177	28691	1410947
Latin America and the Caribbean	13081	15215	16769	15427	13678	15171	21179	28471	21924	25060	...	24747	24676	26011	26547	26867	28818	27856	27173	24950	765148
Northern America	9378	10030	9074	7100	6661	6543	7074	7705	6469	6790	...	8394	9613	9463	10190	8995	8142	7677	7892	8503	241142
Oceania	1942	1839	1675	1018	878	920	904	1200	1181	1539	...	1585	1473	1693	1834	1860	1834	1548	1679	1775	55174

Df_continents, has six rows each representing a continent with 35 columns representing the years from 1980 to 2013 plus the cumulative sum of immigration for each continent.

Now, we're ready to start creating our pie chart with plot() and **kind equals pie**.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
df_continents['Total'].plot(kind='pie')
plt.title('Immigration to Canada by
Continent [1980-2013]')
plt.show()
```

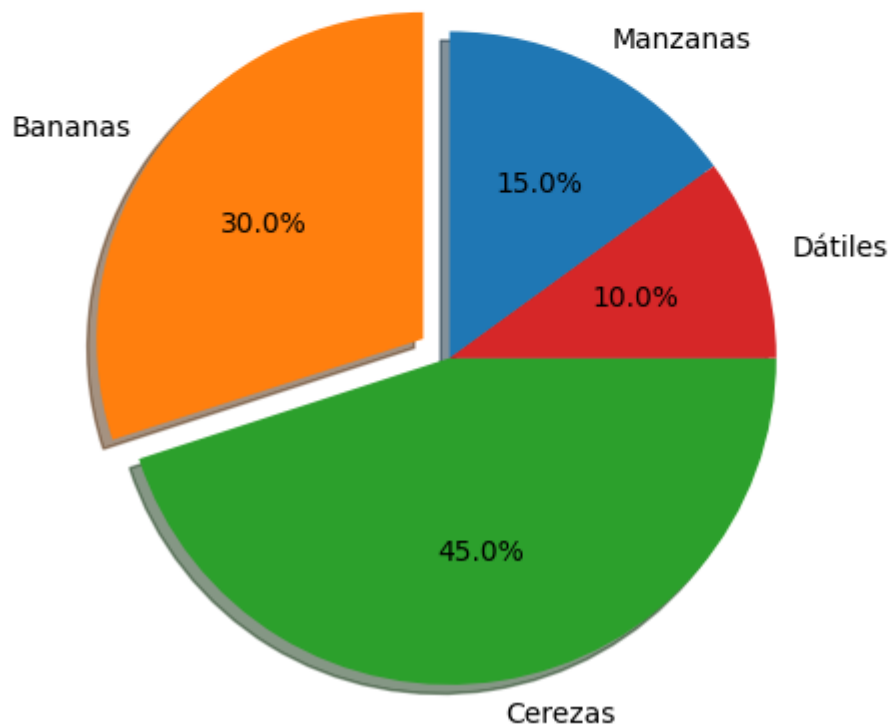


The explode property of a pie chart in Matplotlib Allows you to offset one or more slices from the center In specific sections of the chart by assigning values to the explode parameter. You can control the degree of separation And emphasize particular segments of the pie.


```

1  import matplotlib.pyplot as plt
2
3  # Datos de ejemplo
4  labels = ['Manzanas', 'Bananas', 'Cerezas', 'Dátiles']
5  sizes = [15, 30, 45, 10]
6  explode = [0, 0.1, 0, 0] # "Bananas" se resaltará
7
8  # Crear el gráfico
9  plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%',
10 | | | | shadow=True, startangle=36)
11
12 plt.axis('equal') # Para que el gráfico sea perfectamente circular
13 plt.show()

```



When plotting a pie chart using Matplotlib, `plt.axis('equal')` ensures that the chart is displayed as a **perfect circle** instead of a potentially **distorted ellipse**.

A final point about pie charts there are some strong critics who oppose using pie charts in any condition. They argue that pie charts do not display accurate data consistently when it comes to depicting data consistently and communicating the point.

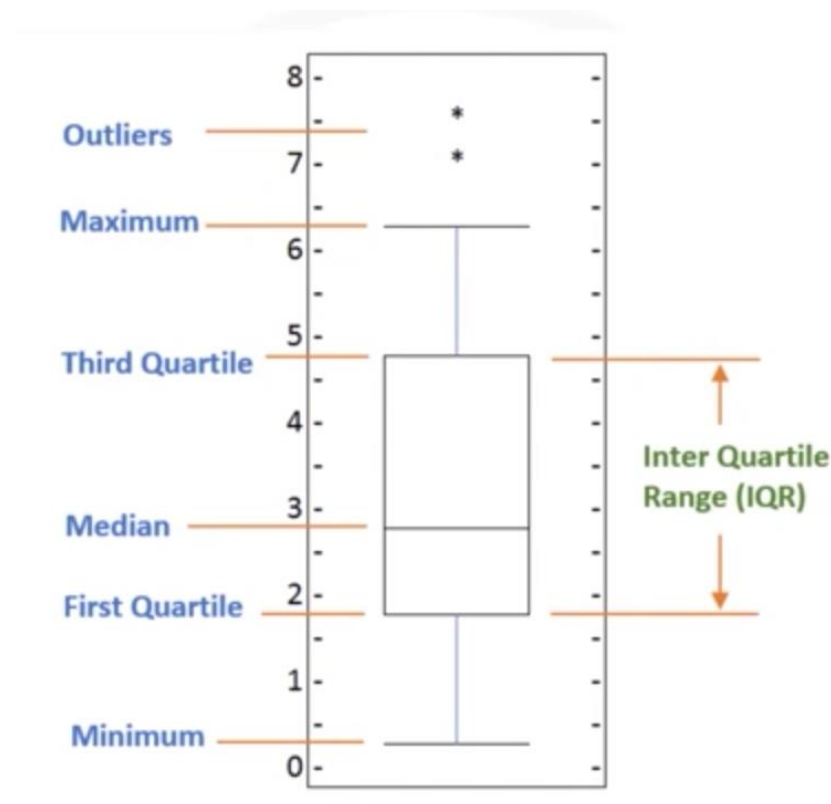
Bar charts perform significantly better.

<https://www.alchemer.com/resources/blog/pie-chart-or-bar-graph/>

Box Plots

A box plot is a way of statistically representing the distribution of given data through five primary dimensions:

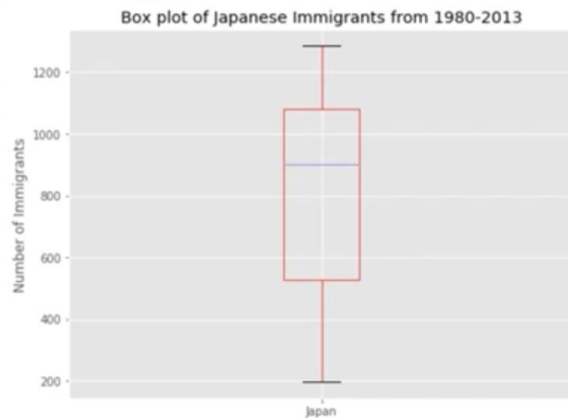
- **Minimum** is usually the value obtained by subtracting 1.5 times the interquartile range, or IQR, from the first quartile.
- **First quartile** is the point 25% of the way through the sorted data. In other words, a quarter of the data points are **less than this value**.
- **Median** is the median of the sorted data.
- **Third quartile** is the point 75% of the way through the sorted data. In other words, three quarters of the data points are **less than this value**.
- **Maximum** is usually the value obtained by adding the third quartile to 1.5 times the IQR.



Finally, box plots also display outliers as individual dots outside the outer and lower extremes.

Let's see how we can create a box plot with Matplotlib.

```
df_japan = df_canada.loc[['Japan'], years].transpose()
df_japan.plot(kind='box')
plt.title('Box plot of Japanese Immigrants from 1980-2013')
plt.ylabel("Number of Immigrants")
plt.show()
```



We first process the data frame, `df_canada`, for Japan. we exclude the total column using the years variable. Then we transpose the resulting dataframe in the correct format to create the box plot.

From this plot, we can verify that there **are no outliers** in this data. Also, we can see that the **median is closer to the top**, indicating **more data concentration in the upper half**.

Recap

In this video, you learned that:

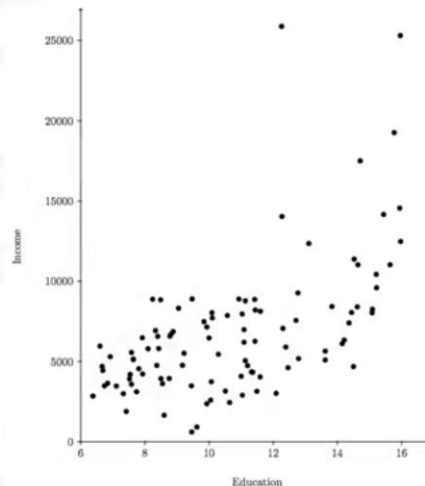
- A box plot is a way of statistically representing given data distribution through five main dimensions.
- The five main dimensions are minimum, first quartile, median, third quartile, and maximum.
- You can create a box plot using Matplotlib.

Scatter Plots

A scatter plot is a type of plot that displays values pertaining to typically two variables against each other.

Usually, it's a dependent variable that is plotted against an independent variable to determine if any correlation between the two variables exist.

- Displays values pertaining to two variables
- Determines the correlation between the two variables



For example, here's a scatter plot of income versus education.

And by looking at the plotted data, one can conclude that an individual with more years of education is likely to earn a higher income than an individual with fewer years of education.

How can we create a scatter plot with matplotlib?

Let's say we want to create a scatter plot of the total annual immigration to Canada from 1980-2013.

To be able to do that, we first need to create a new dataframe that shows each year and the corresponding total number of immigrants from all countries worldwide as shown here.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
df_total.plot(
    kind='scatter',
    x='year',
    y='total',
)

plt.title('Total Immigrant population to Canada from 1980 - 2013')
plt.xlabel('Year')
plt.ylabel('Number of Immigrants')

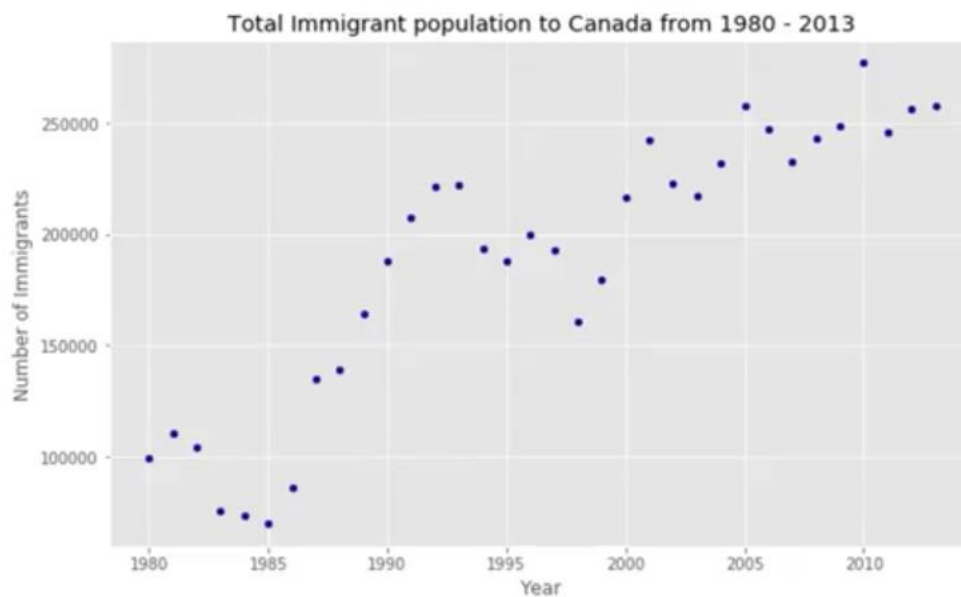
plt.show()
```

df_total	
year	total
1980	99137
1981	110563
1982	104271
1983	75550
1984	73417
.	.
.	.

Let's name this new dataframe as df_total.

We set kind=scatter. We need to pass the variable, which is on the horizontal axis as the x parameter and the variable that is on the vertical axis as the y parameter.

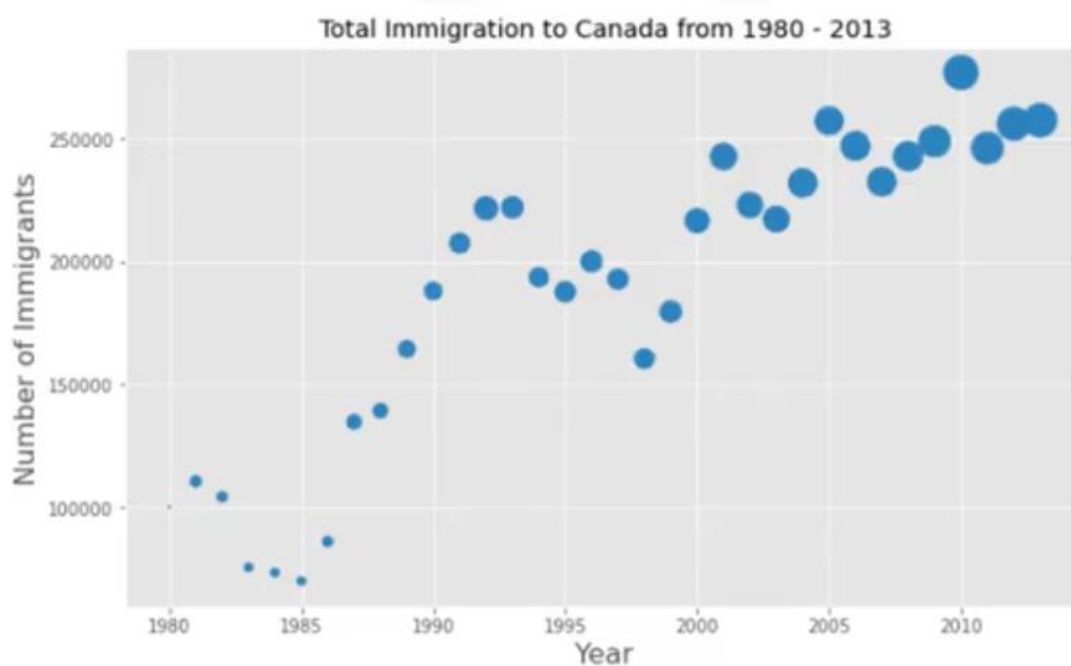
There you have it. A scatter plot that shows total immigration to Canada from countries all over the world from 1980-2013.



The scatter plot clearly depicts an **overall rising trend** of immigration with time.

Consider the use of the color parameter,

You can also use the s parameter to **represent any third variable** if you want to.



Here we have included the total from the African continent to represent the size of each marker over the years. And it is evident that the number of immigrants has increased over these years as the size of the markers in the scatter plot is increasing.

Recap

In this video, you learned that:

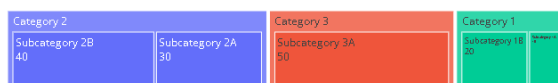
- A scatter plot displays values pertaining to typically two variables against each other.
- The process of creating a scatter plot using Matplotlib to visualize a large set of data.

Understanding Treemaps and Pivot Charts

Treemaps are a form of data visualization that **displays hierarchical data** using nested rectangles. Each branch of the hierarchy is given a rectangle, which is then tiled with smaller rectangles representing sub-branches. Treemaps are particularly useful for visualizing large datasets where the hierarchical structure is crucial, offering **an intuitive and space-efficient** way to display data.

```
1 import pandas as pd
2 import plotly.express as px
3 # Replace with your actual dataset or data source
4 data = {
5     'Category': ['Category 1', 'Category 1', 'Category 2', 'Category 2', 'Category 3'],
6     'Subcategory': ['Subcategory 1A', 'Subcategory 1B', 'Subcategory 2A', 'Subcategory 2B', 'Subcategory 3A'],
7     'Value': [10, 20, 30, 40, 50]
8 }
9 df = pd.DataFrame(data)
10 fig = px.treemap(df,
11                 path=['Category', 'Subcategory'], # Define hierarchical structure
12                 values='Value', # Size of each rectangle
13                 title='Treemap Example', # Title of the treemap
14                 hover_name='Subcategory', # Display subcategory name on hover
15                 hover_data=['Category', 'Subcategory', 'Value'] # Display category, subcategory, and value on hover
16 )
17 fig.update_layout(width=800, height=300) # Adjust the size of the treemap
18 fig.update_traces(textinfo='label+value') # Display label and value on rectangles
19 fig.show()
```

Treemap Example

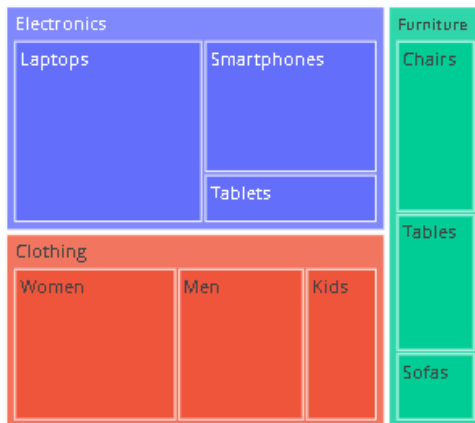


```

data = {
    'Category': ['Electronics', 'Electronics', 'Electronics',
                'Furniture', 'Furniture', 'Furniture',
                'Clothing', 'Clothing', 'Clothing'],
    'Subcategory': ['Laptops', 'Smartphones', 'Tablets',
                   'Chairs', 'Tables', 'Sofas',
                   'Men', 'Women', 'Kids'],
    'Sales': [120000, 80000, 30000,
              50000, 40000, 20000,
              70000, 90000, 40000]
}
df = pd.DataFrame(data)
# Creating the treemap
fig = px.treemap(
    df,
    path=['Category', 'Subcategory'],
    values='Sales',
    title='Sales Data Treemap'
)
fig.update_layout(width=500, height=500) # Adjust the size of the treemap
fig.show()

```

Sales Data Treemap



Pivot charts are a powerful tool used for data visualization and analysis. They allow users to **dynamically summarize** and explore large datasets, revealing insights and trends that might not be immediately obvious. Pivot charts are widely used in business intelligence, finance, marketing, and various other fields where data analysis is crucial.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Define the parameters
num_quarters = 4
num_categories = 4
num_subcategories = 3
num_samples = num_quarters * num_categories * num_subcategories*50

# Generate sample sales data
np.random.seed(40)

# Creating sample data
dates = np.repeat(['Q1','Q2','Q3','Q4'], 50*num_categories * num_subcategories)
categories = np.tile(np.random.choice(['Laptops', 'Desktops', 'Peripherals', 'Software'], size=num_quarters*50), num_categories * num_subcategories)
subcategories = np.tile(np.random.choice(['Accessories', 'Components', 'Software Suites'], size=num_quarters*50), num_categories * num_subcategories)
sales_values = np.random.randint(1000, 5000, size=num_samples)

# Create DataFrame
df = pd.DataFrame({
    'Date': dates,
    'Category': categories,
    'Subcategory': subcategories,
    'Sales': sales_values
})

# Create pivot table
pivot_table = df.pivot_table(index='Date', columns=['Category','Subcategory'], values='Sales', aggfunc='sum')

# Plotting a pivot chart
pivot_table.plot(kind='bar', figsize=(14, 8))
plt.title('Sales Summary of IT Products by Category and Subcategory')
plt.xlabel('Quarters')
plt.ylabel('Total Sales')
plt.grid(False)
plt.legend(title=('Category', 'Subcategory'), bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

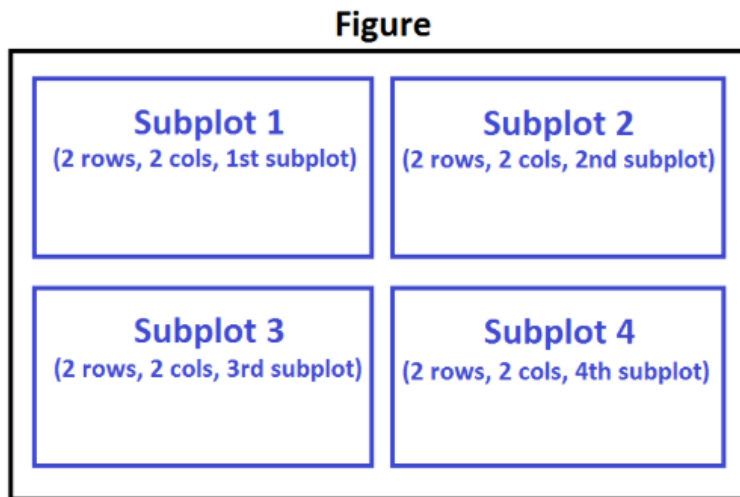
```

Hands-on Lab: Pie Charts, Box Plots, Scatter Plots, and Bubble Plots

Set the country name as index - useful for quickly looking up countries using “.loc” method. That reduces dataset dimensionality in one column.

```
df_can.set_index('Country', inplace=True)
```

Groups of charts



```
fig = plt.figure() # create figure
```

```
ax0 = fig.add_subplot(1, 2, 1) # add subplot 1 (1 row, 2 columns, first plot)
```

```
ax1 = fig.add_subplot(1, 2, 2) # add subplot 2 (1 row, 2 columns, second plot).
```

```
df_Cl.plot(kind='box', color='blue', vert=False, figsize=(20, 6), ax=ax0) # add to subplot 1
```

```
df_Cl.plot(kind='line', figsize=(20, 6), ax=ax1) # add to subplot 2
```

Tip regarding subplot convention

In the case when `nrows`, `ncols`, and `plot_number` are all less than 10, a convenience exists such that a 3-digit number can be given instead, where the hundreds represent `nrows`, the tens represent `ncols` and the units represent `plot_number`. For instance,

```
subplot(211) == subplot(2, 1, 1)
```

produces a subaxes in a figure which represents the top plot (i.e. the first) in a 2 rows by 1 column notional grid (no grid actually exists, but conceptually this is how the returned subplot has been positioned).

Regression line in scatter plot

We will use **Numpy's** `polyfit()` method by passing in the following:

- `x`: x-coordinates of the data.
- `y`: y-coordinates of the data.
- `deg`: Degree of fitting polynomial. 1 = linear, 2 = quadratic, and so on.

```
x = df_tot['year'] # year on x-axis
y = df_tot['total'] # total on y-axis
fit = np.polyfit(x, y, deg=1)
```

The output is an array with the polynomial coefficients, highest powers first. Since we are plotting a linear regression $y = a * x + b$, our output has 2 elements [5.56709228e+03, -1.09261952e+07] with the slope in position 0 and intercept in position 1.

Interesante como en plot() genera valores en el segundo argumento a partir del primero.

```
# plot line of best fit
plt.plot(x, fit[0] * x + fit[1], color='red') # recall that x is the Years
plt.annotate('y={0:.0f} x + {1:.0f}'.format(fit[0], fit[1]), xy=(2000, 150000))
```

Plotting Directly with Matplotlib

- Explore various functions offered by Matplotlib for data visualization and plotting.
- Differentiate between data storytelling and data visualization.

Matplotlib is a general purpose, comprehensive plotting **library** that provides a flexible interface for creating a wide range of plots. Its **pyplot module** offers a convenient way to create and customize plots quickly.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd
```

The subplots function creates a **figure**, the Canvas window, and the **axes**, the area where the plot appears.

```
# Create figure and axes
fig, ax = plt.subplots()
```

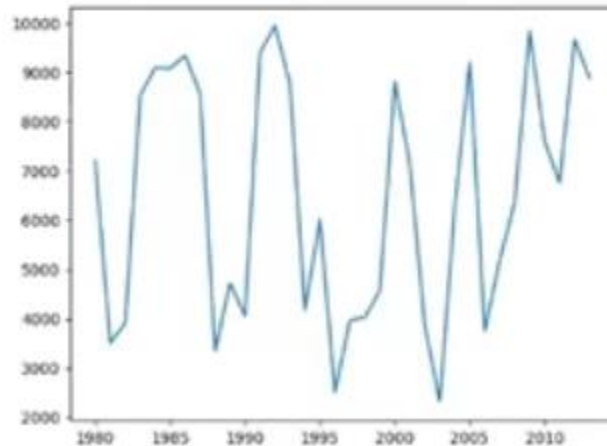
The figure-axes pair provides greater control over the figure or Canvas.

- Line Plot

```
#Create figure and axes
fig, ax = plt.subplots()

# Plot the Line
ax.plot(years, immigrants)

# Display the plot
plt.show()
```



ax.plot function will create a line plot. It takes the x (years) and y (immigrants) values as arguments corresponding to the data to be considered for the x-axis and the y-axis.

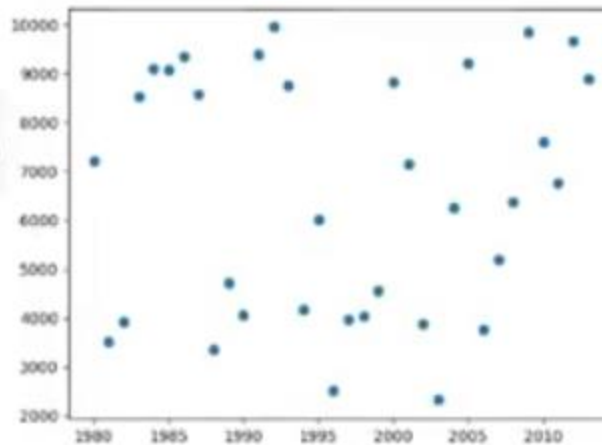
plt.show function display the plot.

- Scatter Plot

```
#Create figure and axes
fig, ax = plt.subplots()

# Plot the Line
ax.scatter(years, immigrants)

# Display the plot
plt.show()
```

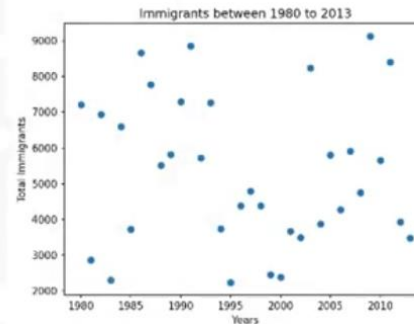
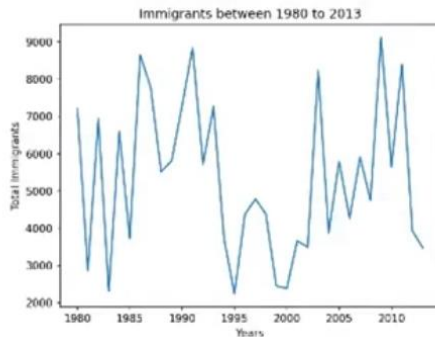


ax.scatter function display this data as a scatter.

Customizing attributes

```
#add title
plt.title('Immigrants between 1980 to 2013')
#add labels
plt.xlabel('Years')
plt.ylabel('Total Immigrants')
```

```
ax.set_title('Immigrants between 1980 to 2013')
ax.set_xlabel('Years')
ax.set_ylabel('Total Immigrants')
```

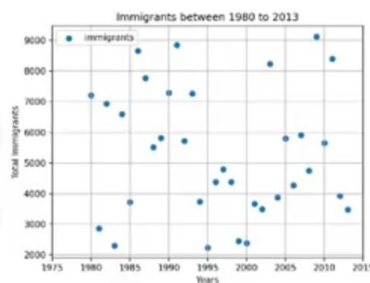
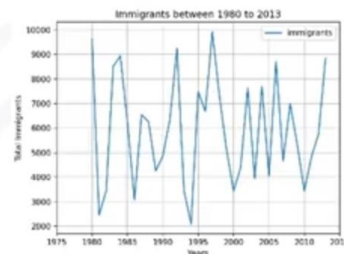


Customizing attributes

```
#limits on x-axis
plt.xlim(1975, 2015)
#or ax.set_xlim()

#Enabling Grid
plt.grid(True)
#or ax.grid()

#Legend
plt.legend(['Immigrants']) #or ax.legend()
```

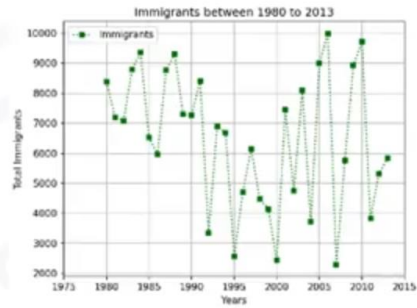


There are various options available to customize the plot.

- line styles: “solid”, “dashed”, or “dotted”.
- marker styles: ‘S’ for square and ‘O’ for dots. marker size.
- color customization

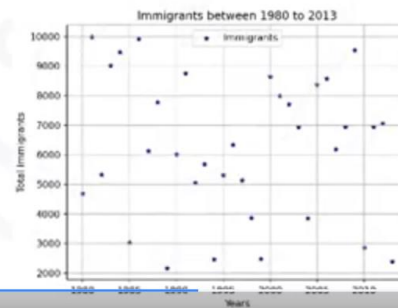
Marker styles, colors, and sizes

```
# Customizing the appearance of Plot
ax.plot(years, immigrants,
        marker='s',
        markersize=5,
        color='green',
        linestyle="dotted")
```



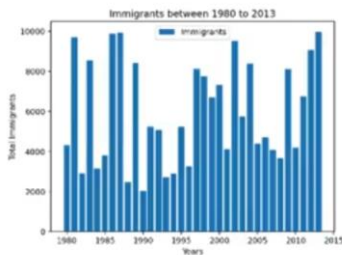
```
# Customizing Scatter Plot
ax.scatter(years, immigrants,
          marker='*',
          s = 20,
          color='darkblue')

# Legend at upper center of the figure
ax.legend(["Immigrants"], loc='upper center')
```

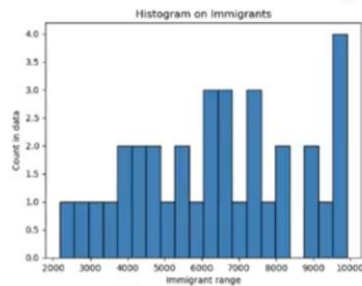


Notice the use of LOC in the legend parameter to specify the location where the legend is placed on the figure.

```
# Plot the bar
ax.bar(years, immigrants)
```

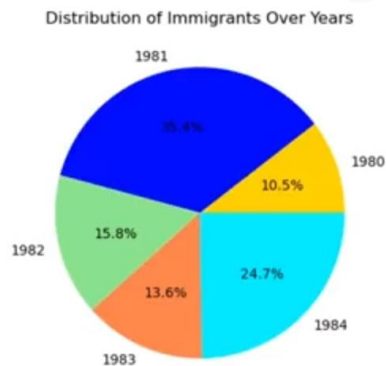


```
# histogram on immigrants
ax.hist(immigrants, bins=20, edgecolor='black', color='steelblue')
```



Notice the use of **edge color** and **color** parameters to specify the border and bar column of the histogram.

```
#Pie on immigrants
ax.pie(immigrants[0:5], labels=years[0:5],
      colors = ['gold','blue','lightgreen','coral','cyan'],
      autopct='%1.1f%%')
```



Colors you are assigned to each pie slice has a list of colors and labels are set is years.

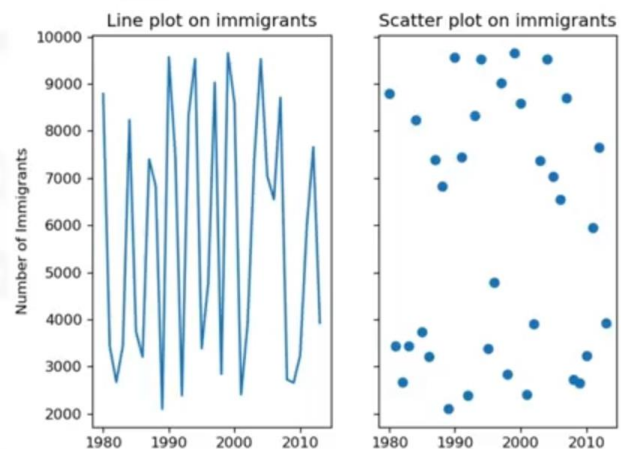
Autopct displays the percentage of immigrants with one decimal point.

Multiple plots and sub-plotting

```
# Create a figure with two axes in a row
fig, axs = plt.subplots(1, 2, sharey=True)
```

```
#Plotting in first axes - the left one
axs[0].plot(years, immigrants)
axs[0].set_title("Line plot on immigrants")
```

```
#Plotting in second axes - the right one
axs[1].scatter(years, immigrants)
axs[1].set_title("Scatter plot on immigrants")
```



`plt.subplots` and pass 1 (number of rows) and 2 (number of plots in a row).

Both the subplot will be sharing the same y-axis as the data in the y-axis is the same, so assign the **Shery** parameter as true.

Now you have two axes, `axs` with index zero `ax[0]` and `axs` with index one, `ax[1]` and you can plot your plots as you did earlier.

Alternatively, we can use the **add_subplot** function. It takes three arguments:

- The number of rows.

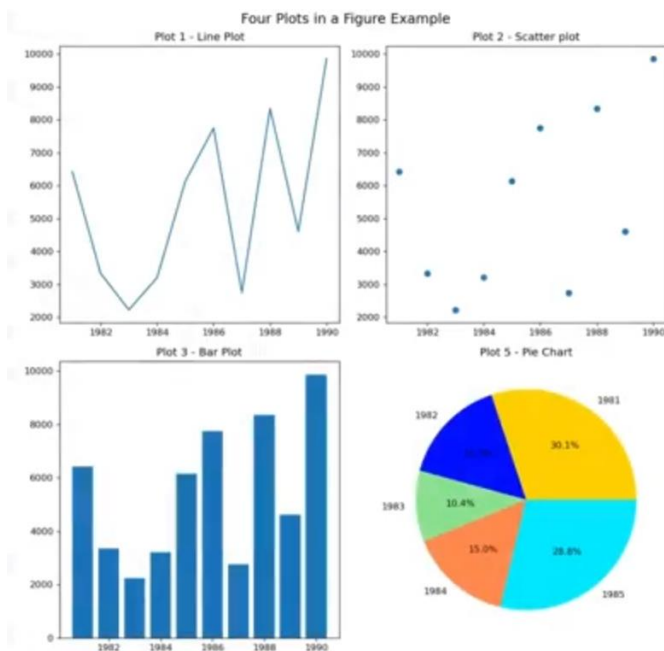
- The number of columns.
- The index of the subplot.

```
# Add the first subplot (top-left)
axs1 = fig.add_subplot(2, 2, 1)
#Plotting in first axes - the left one
axs1.plot(total_immigrants)
axs1.set_title("Plot 1 - Line plot")
```

```
# Add the second subplot (top-right)
ax2 = fig.add_subplot(2, 2, 2)
ax2.scatter(years, immigrants)
ax2.set_title('Plot 2 - Scatter plot')
```

```
# Add the third subplot (bottom-left)
ax3 = fig.add_subplot(2, 2, 3)
ax3.bar(years, immigrants)
ax3.set_title('Plot 3 - Bar Plot')
```

```
# Add the fourth subplot (bottom-right)
ax4 = fig.add_subplot(2, 2, 4)
ax4.pie(immigrants[0:5], labels=years[0:5],
       colors = ['gold', 'blue', 'lightgreen', 'coral', 'cyan'],
       autopct='%1.1f%%')
ax4.set_aspect('equal')
ax4.set_title('Plot 5 - Pie Chart')
```



Data storytelling and data visualization

- Data storytelling
 - Is the art of storytelling
 - Creates a narrative around the data
- Data visualization
 - Is an important aspect of data storytelling
 - Creates engaging visuals

These are **two different terms** and serve different purposes.

Data storytelling is the art of storytelling that involves creating a **narrative** around the data. It presents a **compelling** and **engaging story**.

On the other hand, data visualization is an important aspect of data storytelling and evolves, creating **informative charts** to understand and **explore patterns, trends, and relationships** within the data.

<https://www.forbes.com/sites/brentdykes/2016/03/31/data-storytelling-the-essential-data-science-skill-everyone-needs/>

Recap

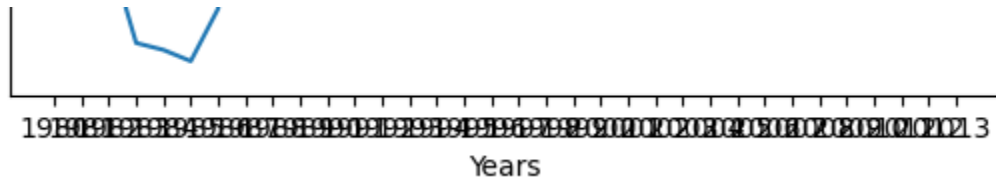
In this video, you learned that:

- Matplotlib is a versatile plotting library that offers a flexible interface for creating various types of plots.
- Matplotlib's pyplot module offers a convenient way to create and customize plots quickly.
- Data Storytelling is the 'art of storytelling' that involves creating a narrative around the data.
- Data visualization is an important aspect of data storytelling and involves creating engaging visuals.

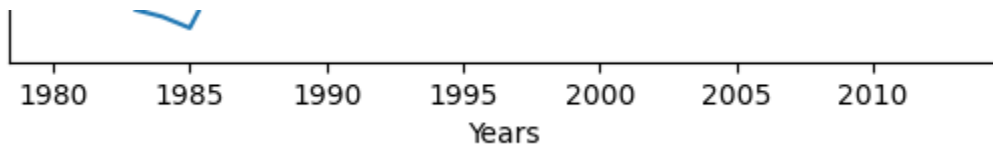
Hands-on Lab: Plotting Directly with Matplotlib

#Changing the index type to integer

`total_immigrants.index = total_immigrants.index.map(int)` is importante para que el eje X trata correctamente los valores y en lugar de salir un churro



Salga un eje x chulo



The `hist` function returns list of arrays with 1. counts and 2. bins. we can fetch that using unpacking functionality

```
count = ax.hist(df_country['2013'])
```

count

[	178.,	11.,	1.,	2.,	0.,	0.,	0.,	0.,	1.,	2.],	
[	0.,	3412.9,	6825.8,	10238.7,	3651.6,	17064.5,	20477.4,	23890.3,	27303.2,	30716.1,	34129.)]

Practice Quiz: Specialized Visualization Tools

Module 2 Summary: Basic and Specialized Visualization Tools

- A pie chart is a circular statistical graphic, divided into segments, to illustrate numerical proportion.
- The process of creating a pie chart involves importing Matplotlib to represent a large set of data over a period of time.
- A box plot is a way of statistically representing given data distribution through five main dimensions.
- The five main dimensions are minimum, first quartile, median, third quartile, and maximum.
- You can create a box plot using Matplotlib.

- A scatter plot displays values pertaining to typically two variables against each other.
- The process of creating a scatter plot involves importing Matplotlib to visualize a large set of data.
- Matplotlib is a versatile plotting library that offers a flexible interface for creating various types of plots.
- Matplotlib's Pyplot module offers a convenient way to create and customize plots quickly.
- Data Storytelling is the 'art of storytelling' that involves creating a narrative around the data.
- Data visualization is an important aspect of data storytelling and involves creating engaging visuals.

Module 2 Cheat Sheet

Module 2 Graded Quiz: Basic and Specialized Visualization Tools

Module 3

Title: Advanced Visualizations and Geospatial Data

Description

Advanced visualization tools are sophisticated platforms that provide a wide range of advanced features and capabilities. These tools provide an extensive set of options that help create visually appealing and interactive visualizations. In this module, you will learn about waffle charts and word cloud including their application. You will explore Seaborn, a new visualization library in Python, and learn how to create regression plots using it. In addition, you will learn about folium, a data visualization library that visualizes geospatial data. Furthermore, you will explore the process of creating maps using Folium and superimposing them with markers to make them interesting. Finally, you will learn how to create a Choropleth map using Folium.

Objectives

By the end of this week, you will be able to:

- Explore waffle charts and word cloud along with their application
- Describe Seaborn and explore the process of generating attractive regression plots
- Describe Folium and explore the process of creating maps
- Explore the process of superimposing markers on maps using Folium
- Describe Choropleth maps with the help of an illustration
- Explore the process of creating a Choropleth map using Folium

Activities

Lesson 1: Advanced Visualizations and Geospatial Data

Waffle Charts & Word Cloud

What you will learn



Explore waffle charts with the help of an illustration



Identify the use cases of waffle charts



Explore word cloud with the help of an illustration



Identify the use cases of word cloud

Waffle charts are a visualization technique that represent categorical data in the form of square tiles or cells. These resemble a grid of equal-sized squares, with each square representing a specific value or category. The size or color of the squares indicate the magnitude or proportion of each category.

Waffle charts effectively show the **proportion or percentage of different categories within an overall composition**. The grid-like structure of waffle charts makes it **easy to understand** and interpret data even for **nontechnical audiences**.

Waffle charts

- Represents categorical data in the form of:
 - Square tiles
 - Cells
- Displays proportion or percentage of different categories
- Simplifies data for all types of audiences



Waffle charts: Use case

You can use waffle charts for:

- Market share analysis
- Demographic representation
- Project progress tracking
- Budget allocation
- Survey responses
- Election results
- Product sales analysis

- Visualize market share data, showing the **proportion** of companies or products **within** a specific **market**.
- Display demographic data, such as age **groups** or ethnicities **within** a **population**.
- Represent completion status of tasks or milestones, providing a visual **overview of progress**.
- Demonstrate allocation of budgetary **resources** across categories or departments **within** an **organization**.
- Summarize survey responses, displaying the **distribution of answers** to multiple choice questions.
- Provide a clear visualization of voting outcomes, representing the **distribution of votes** for candidates or parties.
- illustrate product sales by representing the **distribution** of **sales across categories** or regions.

Use the pywaffle library in Python, to create visually appealing waffle charts to effectively communicate categorical data.

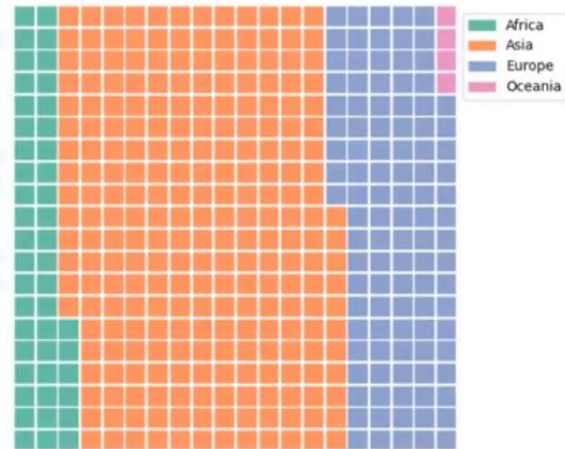
pywaffle library

```
import matplotlib.pyplot as plt
from pywaffle import Waffle

# Create data for the waffle chart
data = df_dsn[['Africa', 'Asia', 'Europe', 'Oceania']]

# Set up the waffle chart figure
fig = plt.figure(
    FigureClass=Waffle,
    rows=20,
    columns=20,
    values=data,
    legend={'labels': ['Africa', 'Asia', 'Europe', 'Oceania'],
           'loc': 'upper left', 'bbox_to_anchor': (1, 1)})

# Display the chart
plt.show()
```



The **rows** and **columns** parameters determine the size of the chart grid.

Word cloud, also known as **tag cloud** or **text cloud**, is a popular data visualization method to visually **present textual data** in an engaging and informative manner.

It presents a concise **summary of the textual content** by providing a visual overview of the **most commonly used words** within a given text or collection of documents.

A word cloud is simply a depiction of the **importance** of different words in a body of text.

It works in a simple way, a word **appears bigger and bolder** in the word cloud depending on how **many times it appears** in a source of textual data.

Word cloud

Bandusky 2

After leaving Reavis, I must attend college because it is definitely a requirement for becoming a veterinarian. In fact, a bachelor's degree is necessary in order to even enter a veterinarian program. One must also possess excellent communication, leadership, public speaking, and organizational skills. I have put a lot of thought and consideration into college, and I have decided that I would like to go to the University of Illinois. It is a wonderful school, and they even have a graduate program designed for students who want to become veterinarians.

Once I have completed a veterinarian program, I will be able to pursue my dream career. This career provides numerous benefits, the first of which is salary. The average veterinarian salary is \$60,000 a year, a salary that would definitely allow me to live a comfortable life. Secondly, it is a rewarding job. This job would provide me with the satisfaction of knowing that I am helping or saving an animal's life. Finally, becoming a veterinarian would assure me a lifetime of happiness. I know I would love going to my job every day, because I would be working with what I love most: animals.

In summary, when I graduate from Reavis, I plan to go to college to become a veterinarian. I love animals and I want to do anything that I can to help them. I know I am only a freshman, but I also know that I am growing up quickly. As Ferris Bueller quotes, "Life moves pretty fast. If you don't stop and look around once in a while, you could miss it!"



Let's now explore the areas in which we can employ word cloud.

Word cloud: Use case

You can use word cloud for:

- Social media analysis
 - Customer feedback analysis
 - Content analysis
 - Market research
 - Resume or job description analysis
-
- Extract and visualize **popular topics** or sentiments from social media conversations or hashtags.
 - Summarize customer reviews or feedback to identify **recurring** themes or **sentiments**.
 - Analyzing textual content, such as articles, blogs, or research papers to **uncover prevalent keywords** or themes.
 - Analyze survey responses, interviews, or focus group transcripts to **extract key insights**.
 - Highlights important skills or **keywords** in resumes or job descriptions to assess their **relevance**.

Recap

In this video, you learned that:

- Waffle charts are a visualization technique that represent categorical data in the form of square tiles or cells.
- There are different areas in which you can use waffle charts.
- Word cloud is a popular data visualization method to visually present textual data in an engaging and informative manner.
- You can use word cloud in different areas to visually present textual data in an engaging and informative manner.

Seaborn and Regression Plots

What you will learn



Although Seaborn is another data visualization **library**, it's **based** on **Matplotlib**.

Seaborn offers a range of built-in themes and color palettes that **improve** the **visual appeal** of your plots with minimal effort.

Seaborn makes creating plots very efficient, therefore, with Seaborn, you can generate plots with **code that is five times less** than with Matplotlib.

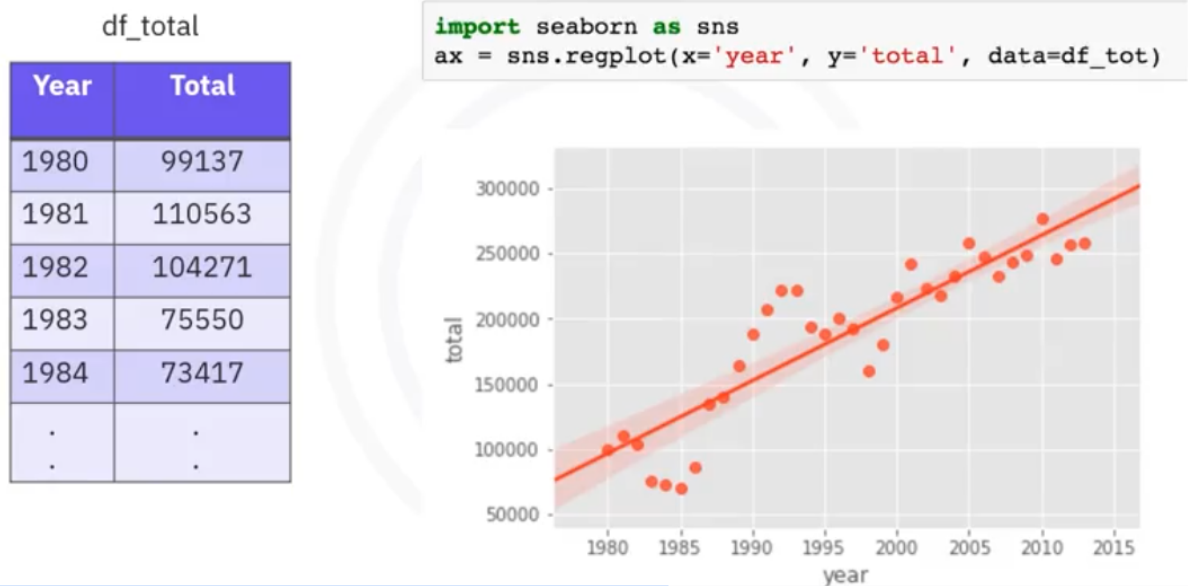
Seaborn integrates well with statistical libraries such as NumPy and SciPy, allowing you to easily **combine statistical analysis with visualizations**.

It provides specialized plot types such as regression plots, distribution plots, and categorical plots, that are particularly **useful** for analyzing data and **modeling relationships**.

While Pandas and Matplotlib are powerful tools for data manipulation and basic visualization, Seaborn complements them by providing a higher level interface for creating visually appealing and informative statistical graphics.

Seaborn works well, especially when dealing with more complex visualizations and statistical analyses.

Regression Plots



The output of this one line of code is a scatter plot with a regression line, and not just that, but also a 95% confidence interval.

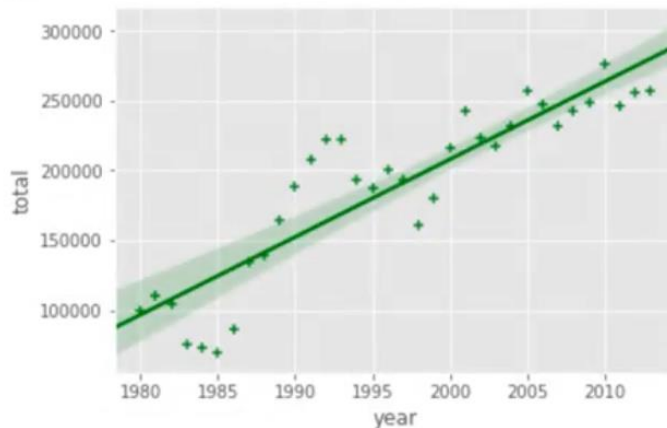
Parameters to customize color, marker shape

Regression Plots

df_total

Year	Total
1980	99137
1981	110563
1982	104271
1983	75550
1984	73417
.	.
.	.

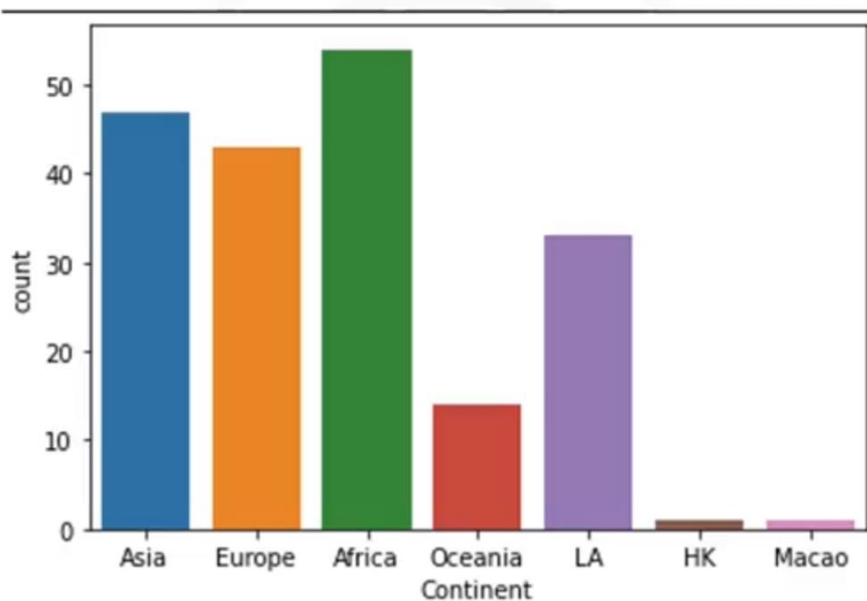
```
import seaborn as sns
ax = sns.regplot(x='year', y='total', data=df_tot,
                 color='green', marker='+')
```



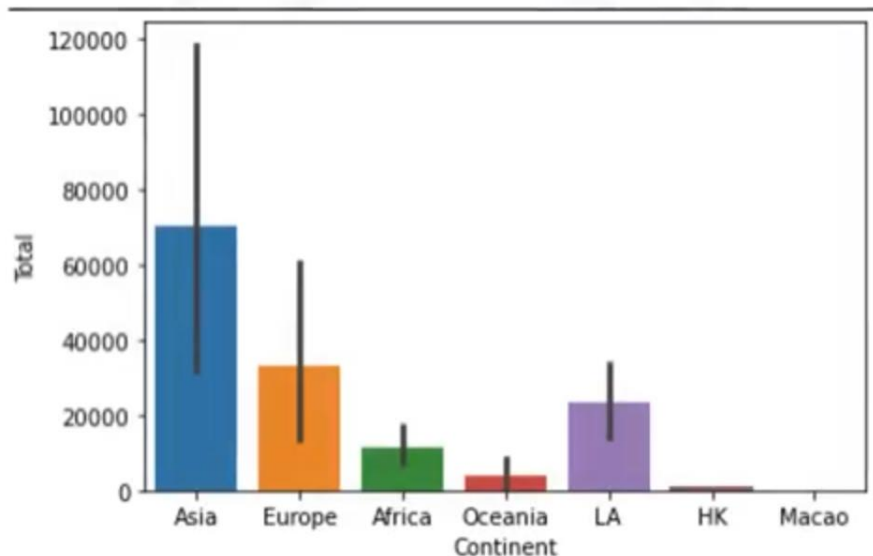
Let's try to plot some categorical data.

Categorical Plots

```
sns.countplot(x='Continent', data=df_canada)
```



```
sns.barplot(x='Continent',y='Total',data=df_canada)
```



Recap

In this video, you learned that:

- Seaborn is a Data Visualization library based on Matplotlib.
- Seaborn was built primarily to provide a high-level interface for drawing statistical graphics.
- Scatter Plots and Regression Lines can be created with one line of code using Seaborn.
- Seaborn's Regplot function accepts additional parameters for personal customization.

Hands-on Lab: Waffle Charts, Word Clouds, and Regression Plots

Hay varios estilos

```
mpl.style.use('ggplot') # optional: for ggplot-like style
```

```
print(plt.style.available)
```

```
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh',  
'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'petroff10',  
'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark',  
'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-  
muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-  
v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-  
v0_8-whitegrid', 'tableau-colorblind10']
```

Hya stylos personalizables en un fichero

A custom style file (my_custom_style.mplstyle) might look like this:

```
# my_custom_style.mplstyle  
axes.facecolor: "#EAEAF2"  
axes.edgecolor: "#C2C2C2"  
axes.labelcolor: "#333333"  
axes.grid: True  
grid.color: "#FFFFFF"  
grid.linestyle: "-"  
grid.linewidth: 1.0  
text.color: "#333333"  
xtick.color: "#333333"  
ytick.color: "#333333"  
figure.facecolor: "#FFFFFF"  
savefig.facecolor: "#FFFFFF"  
font.size: 12  
lines.linewidth: 2  
lines.color: "#4C72B0"  
  
# Assuming 'my_custom_style.mplstyle' is in a discoverable path  
plt.style.use('my_custom_style')  
fig, ax = plt.subplots()  
ax.plot(np.arange(10), np.random.rand(10) * 10, label='Custom Style')  
ax.set_title('Custom Style')  
ax.legend()  
plt.show()
```

Nube de palabras en una mascara

What you will learn



Describe Folium and its features



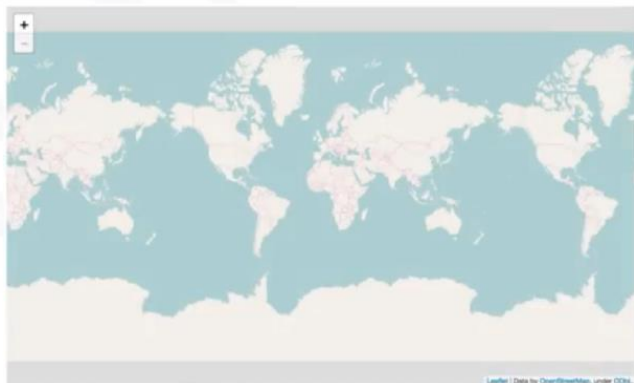
Explain what is Folium used for

Folium is a powerful data visualization **library** in Python that was built primarily to help people **visualize geospatial data** in any location in the world using latitude and longitude values.

You can also create a map and **superimpose markers and clusters on top** of the map for interesting visualizations, you can also create maps of different styles, such as street level maps, stamen maps, and a couple of others

```
#import library  
import folium
```

```
# define the world map  
world_map = folium.Map()  
  
# display world map  
world_map
```



What's interesting about the maps created by Folium is that they are **interactive**, so you can **zoom in and out** after the map is rendered, which is a helpful feature.

The default map **style** is **open street map**, which shows a straight view of an area when you are zoomed in and shows the borders of the world countries when you are zoomed out all the way.

```
Graus = [41.400559201859785, 2.1217400825983037]  
Graus_map = folium.Map(location = Graus, zoom_start = 12)
```

Map centered around “location” at certain zoom level.

Map styles: Stamen Toner

```
# create a Stamen Toner map of  
# the world centered around Canada  
world_map = folium.Map(  
    location=[56.130, -106.35],  
    zoom_start=4,  
    tiles='Stamen Toner'  
)  
  
# display map  
world_map
```



Map styles: Stamen Terrain

```
# create a Stamen Toner map of  
# the world centered around Canada  
world_map = folium.Map(  
    location=[56.130, -106.35],  
    zoom_start=4,  
    tiles='Stamen Terrain'  
)  
  
# display map  
world_map
```



different styles,

tiles="Stamen Watercolor"

tiles="Stamen Toner"

Maps with Markers

What you will learn



Explain how Folium can add markers to a map



Describe how to generate markers

With Folium you can easily **add markers** on the map.

Remember that the **location** parameter **specifies** the latitude and longitude coordinates of the **center point of the map**.

Markers play a vital role in **enhancing interactivity** and adding context to maps. They represent specific locations or points of interest, providing **additional information when clicked**.

Markers are like signposts that guide us through the map, highlighting important elements.

```
import folium

# Create a map object centered at Canada
canada_map = folium.Map(location=[56.1304, -106.3468], zoom_start=4)

# Display the map with the marker
canada_map

# Add a marker for Ontario province
folium.Marker(location=[51.2538, -85.3232], popup='Ontario').add_to(canada_map)
```

The **add_to**(canada_map) **method** is called on **the folium.Marker object** to add the marker for Ontario to the canada_map. This **ensures** that the **marker is included** as part of the map's layers and will be displayed when the **canada_map is rendered or saved**.

Alternatively, we can create the marker using FeatureGroup.

Add marker with feature group

```
# generate map of Canada
canada_map = folium.Map(
    location=[56.130, -106.35],
    zoom_start=4
)

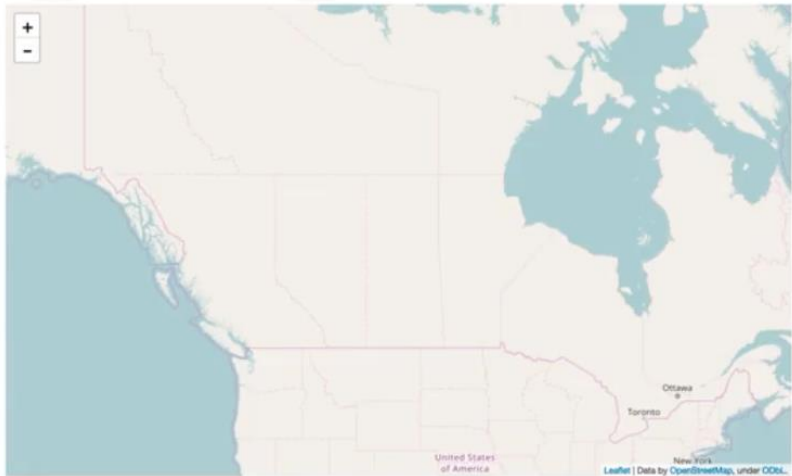
## add a red marker to Ontario

# create a feature group
ontario = folium.map.FeatureGroup()

# style the feature group
ontario.add_child(
    folium.features.CircleMarker(
        [51.25, -85.32], radius = 5,
        color = "red", fill_color = "Red"
    )
)

# add the feature group to the map
canada_map.add_child(ontario)

# display map
canada_map
```



Now, when a **feature group** is created [`folium.map.FeatureGroup()`], it is **empty**, so what's next is to start **creating** what is called **children** and adding them to the feature group [`add_child()`] in the form of a red **circular** mark [`folium.features.CircleMarker()`] located at the center of the Ontario province.

We specify the location of the child by passing in its latitude and longitude values. Once we're done adding children to the feature group, we add the feature group to the map [`add_child()`].

Label the marker

```
# generate map of Canada
canada_map = folium.Map(
    location=[56.130, -106.35],
    zoom_start=4
)

## add a red marker to Ontario

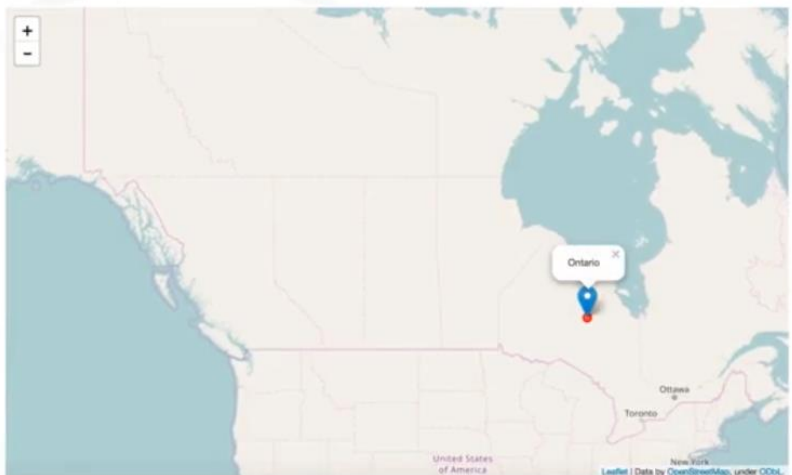
# create a feature group
ontario = folium.map.FeatureGroup()

# style the feature group
ontario.add_child(
    folium.features.CircleMarker(
        [51.25, -85.32], radius = 5,
        color = "red", fill_color = "Red"
    )
)

# add the feature group to the map
canada_map.add_child(ontario)

# label the marker
folium.Marker([51.25, -85.32],
    popup='Ontario').add_to(canada_map)

# display map
canada_map
```



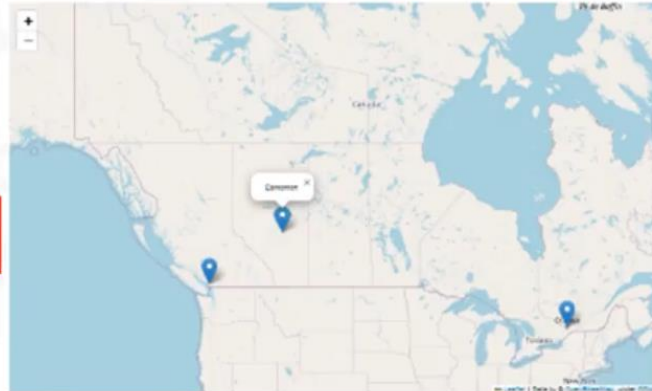
Now, it would be helpful if we could label this marker to let other people know what it represents. To do that, we use the marker function [Marker().add_to()] and the pop-up parameter to pass in the text we want to add to this marker.

Multiple markers

```
# Define a list of locations and their corresponding popups
locations = [
    {"location": [45.4215, -75.6989], "popup": "Ottawa"},
    {"location": [53.5461, -113.4938], "popup": "Edmonton"},
    {"location": [49.2827, -123.1207], "popup": "Vancouver"},
    # Add more locations and their popups here
]

# Add markers for each location in the list
for loc in locations:
    folium.Marker(location=loc["location"],
                  popup=loc["popup"]).add_to(map)

# Display the map with the markers
map
```



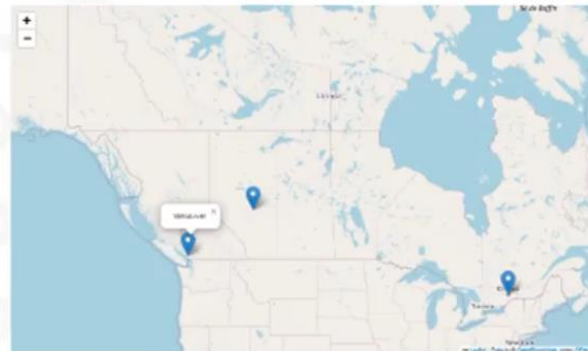
Create a list of all the locations, then pass this list to [Marker().add_to()] through a loop and your map with multiple markers will be ready for display.

Multiple markers

```
#import MarkerCluster
from folium.plugins import MarkerCluster

# Create a MarkerCluster object
marker_cluster = MarkerCluster().add_to(map)

# Add markers for each location in the list to the MarkerCluster
for loc in locations:
    folium.Marker(location=loc["location"],
                  popup=loc["popup"]).add_to(marker_cluster)
```



When creating code to display multiple markers on a map, you can call the **MarkerCluster.add_to()** and pass your map here. Then pass this list to the marker function through the loop. But this time, instead of map, use marker_cluster to add the underscore to functions and your map with multiple markers will be displayed.

The markers within the marker_cluster will be intelligently grouped based on their proximity when the map is displayed. This **clustering feature enhances the visual presentation by preventing overcrowding** and ensuring a clear representation, primarily when numerous markers are close.

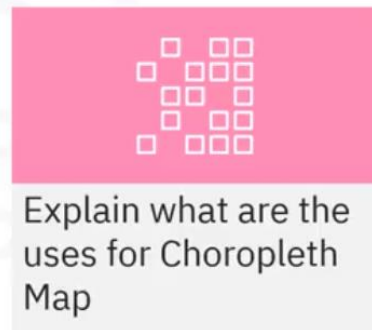
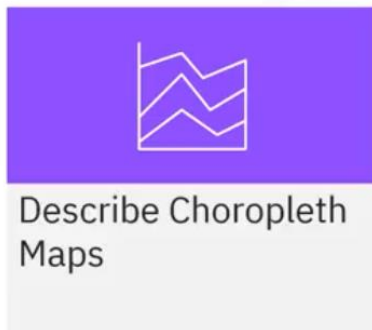
Recap

In this video, you learned that:

- With Folium, you can easily add markers on maps
- The location parameter specifies the latitude and longitude coordinates of the center point of the map
- Markers play a vital role in enhancing interactivity and adding context to maps
- The `folium.Marker()` function specifies location parameters
- The popup parameter provides a label upon being clicked
- Markers can be created using “feature group”

Choropleth Maps

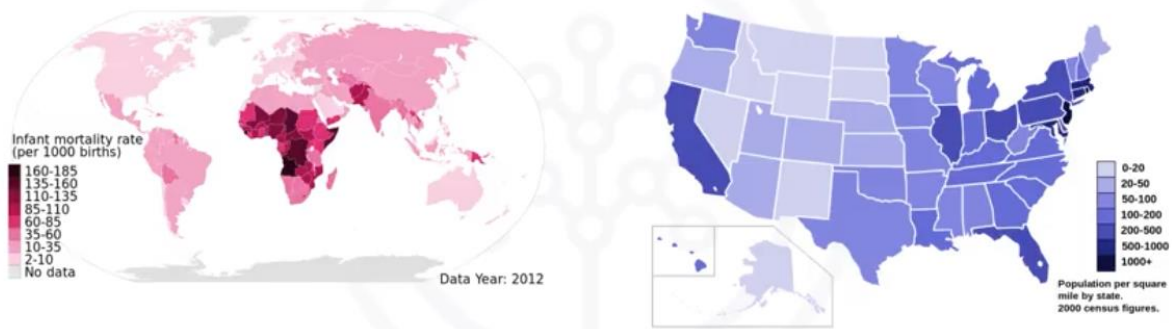
What you will learn



A choropleth map is a **thematic map** in which areas are **shaded** or patterned in **proportion** to the **measurement of the statistical variable displayed** on the map.

The higher the measurement, **the darker** the color.

Choropleth Maps



Folium is a Python library used for creating interactive maps and visualizations. It provides a simple and intuitive way to **generate maps using data from various sources**, including GeoJSON, Pandas DataFrames, and NumPy arrays.

To create a choropleth map of a region of interest, folium requires a **GeoJSON file that includes geospatial data of the region**.

For a choropleth map of the world, we would need a GeoJSON file that lists each country and any geospatial data to define its borders and boundaries.

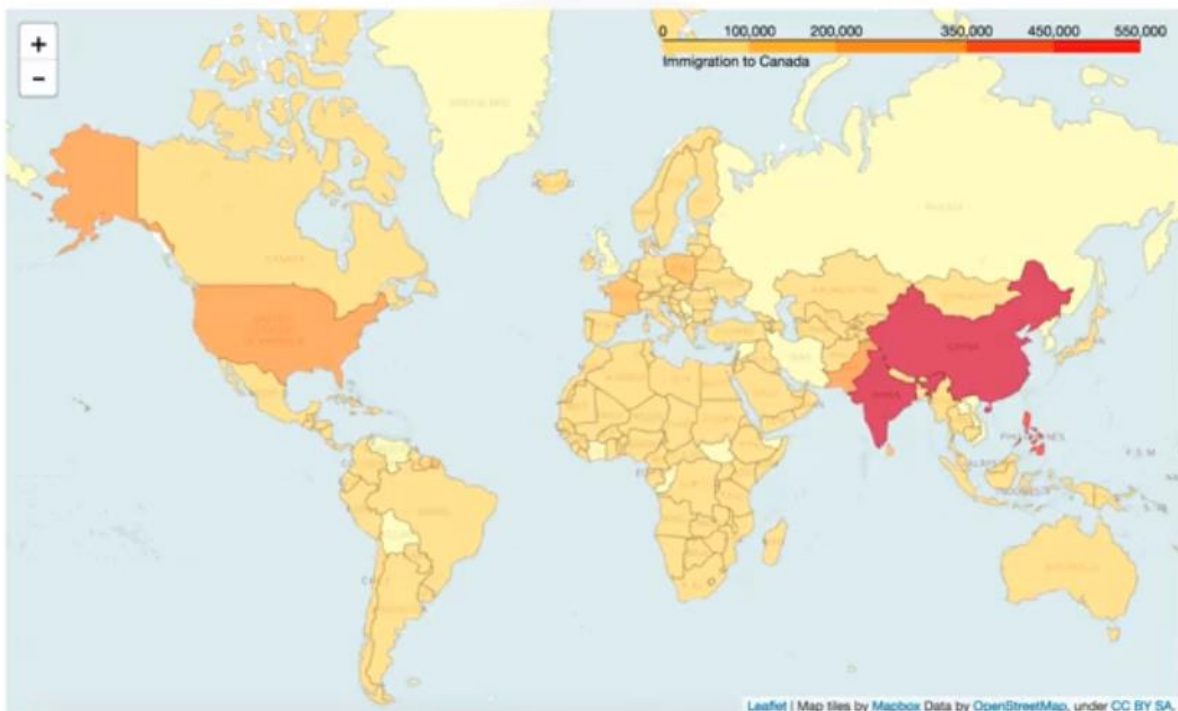
```

{
  "type": "FeatureCollection",
  "features": [{
    "type": "Feature",
    "properties": {
      "name": "Brunei"
    },
    "geometry": {
      "type": "Polygon",
      "coordinates": [
        [
          [114.204017, 4.525874], [114.599961, 4.900011], [115.45071, 5.44773],
          [115.4057, 4.955228], [115.347461, 4.316636], [114.869557, 4.348314],
          [114.659596, 4.007637], [114.204017, 4.525874]
        ]
      ]
    },
    "id": "BRN"
  }],
}

```

As you can see, the file consists of its **name**, **ID**, **geometry**, shape, and **coordinates** that define its borders and boundaries.

So let's see how we can create a choropleth map of the world showing immigration to Canada.



We should be experts now in creating world maps with Folium.

Let's create a world map with “Mapbox Bright” Tileset. The result is a nice **world map displaying the name of every country**. Now, let's convert this map into a choropleth map.

Creating the Map

```
# create a plain world map
world_map = folium.Map(
    zoom_start=2,
    tiles='Mapbox Bright'
)

## geojson file
world_geo = r'world_countries.json'

# generate choropleth map using the total
# population of each country to Canada from
# 1980 to 2013
world_map.choropleth(
    geo_path=world_geo,
    data=df_canada,
    columns=['Country', 'Total'],
    key_on='feature.properties.name',
    fill_color='YlOrRd',
    legend_name='Immigration to Canada'
)

# display map
world_map
```



Define a variable that points to our JSON file. It becomes the argument `geo_path`.

Apply the `choropleth` function to our world map. to use total and country columns names to look up the geospatial information about each country in the GeoJSON file.

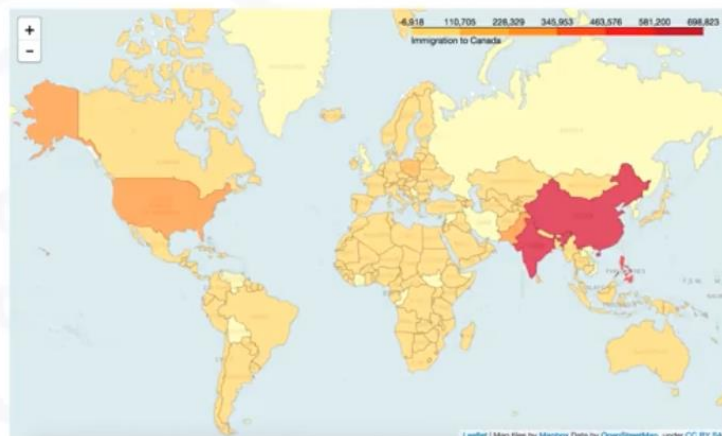
The result will be a choropleth map of Canada showing the intensity of immigration from different countries worldwide.

```
# create a plain world map
world_map = folium.Map(
    zoom_start=2,
    tiles='Mapbox Bright'
)

## geojson file
world_geo = r'world_countries.json'

# generate choropleth map using the total
# population of each country to Canada from
# 1980 to 2013
world_map.choropleth(
    geo_path=world_geo,
    data=df_canada,
    columns=['Country', 'Total'],
    key_on='feature.properties.name',
    fill_color='YlOrRd',
    legend_name='Immigration to Canada'
)

# display map
world_map
```



In this video, you learned that:

- A choropleth map is a thematic map in which areas are shaded or patterned in proportion to the measurement of the statistical variable.
- When creating a choropleth map, Folium requires a GeoJson file that includes geospatial data of the region.
- The Mapbox Bright Tileset displays the name of every country when used on a map.

Hands-on Lab: Creating Maps and Visualizing Geospatial Data

Practice Quiz: Visualizing Geospatial Data

Module 3 Summary: Advanced Visualizations and Geospatial Data

- Folium is a data visualization library in Python that helps people visualize geospatial data.
- With Folium, you can create maps of different styles, such as street-level maps, stamen maps, and more.
- A feature of Folium is that you can create different map styles using the tiles parameter.
- With Folium, you can easily add markers on maps.
- The 'location' parameter specifies the latitude and longitude coordinates of the center point of the map.
- Markers play a vital role in enhancing interactivity and adding context to maps.
- The folium.Marker() function specifies location parameters.
- The popup parameter provides a label upon being clicked.
- Markers can be created using "feature group."
- A choropleth map is a thematic map in which areas are shaded or patterned in proportion to the measurement of the statistical variable.
- When creating a choropleth map, Folium requires a GeoJson file that includes geospatial data of the region.
- The Mapbox Bright Tileset displays the name of every country when used on a map.

Module 3 Cheat Sheet

Module 3 Graded Quiz: Advanced Visualizations and Geospatial Data

Module 4

Title: Creating Dashboards with Plotly and Dash

Description

Dashboards and interactive data applications are crucial tools for data visualization and analysis because they provide a consolidated view of key data and metrics in a visually appealing and understandable format. In this module, you will explore the benefits of dashboards and identify the different web-based dashboarding tools in Python. You will learn about Plotly and discover how to use Plotly graph objects and Plotly express to create charts. You will gain insight into Dash, an open-source user interface Python library, and its two components. Finally, you will gain a clear understanding of the callback function and determine how to connect core and HTML components using callback.

Objectives

By the end of this week, you will be able to:

- Identify different web-based dashboarding tools available in Python
- Explore Plotly and its two sub-modules
- Use Plotly graph objects and Plotly express to create charts
- Discover Dash and its two components
- Describe the callback function
- Determine the process of connecting core and HTML components using callback

Activities

Lesson 1: Creating Dashboards with Plotly and Dash

Dashboarding Overview

What you will learn



Explore the benefits of using interactive data applications to improve business performance



Identify different web-based dashboarding tools available in Python

With real-time visuals on the dashboard, understanding business moving parts becomes easy.

Based on the report type and data, suitable graphs and charts can be created in one central location. This provides an easy way for stakeholders to **understand what is going right or wrong** and what improvements are necessary.

Also, getting the **big picture** in one place can help businesses make **informed decisions** which can improve performance.

Dashboards answer critical business questions.

Let's say you're assigned a task to monitor and report the performance of domestic US flights.

Following are the yearly review report items:

Top 10 airline carriers in the year 2019 in terms of the number of flights.

Number of flights in 2019, split by month.

Number of travelers from California state to other states split by distance group.

Let's look at two ways of presenting the report. Type 1, the report is presented through tables with inference from tables documented for reference.

Type one: Without dashboard

1.

Reporting_Airline	Flights
WN	13972
DL	10241
AA	9853
OO	8515
UA	6430
YX	3416
MQ	3368
B6	3170
OH	2969
AS	2783

2.

Month	Flights
1	6125
2	5578
3	6553
4	6282
5	6441
6	6583
7	6798
8	6753
9	6140
10	6533
11	6280
12	6550

3.

DestState	DistanceGroup	Flights
AK	10	4
AL	8	1
AR	7	1
AZ	2	361
AZ	3	220
AZ	4	9
CA	1	283
CA	2	2256
CA	3	36
CO	3	19
CO	4	401
CO	5	2
CT	11	2
FL	9	51
FL	10	90
FL	11	25
GA	8	114
GA	9	65
HI	10	168
HI	11	116
ID	2	7
ID	3	60
IL	7	169
IL	8	154
IN	8	16
KY	8	13
KY	9	5
LA	7	33
LA	8	14
MA	11	128

Type 2, here, we are presenting the same report in the dashboard format. As you can see, hovering over each chart will provide details about the data points. At the bottom in the sunburst chart, you can click on different numbers, drill down into levels, and get detailed information about each segment.

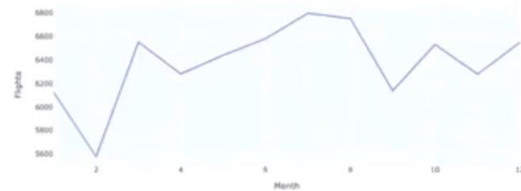
Type two: With dashboard

Airline Dashboard

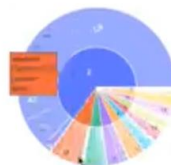
Top 10 airline carrier in year 2019 in terms of number of flights



Number of flights by month - 2019



Number of travelers from CA to other states split by distance group



Can you observe the difference in the presentation of the findings? What if we need to get the report on real-time data, not static data? Also presenting the result using tables and documents is **time-consuming**, **less visually appealing**, and **hard to comprehend**.

A data scientist should have the ability to create and deliver a story around the finding in a way stakeholders can easily understand. With that in mind, dashboards are the way to go.

Let's take a look at web-based dashboarding tool options available in Python.

Web-based dashboarding



- Dash is a Python framework for building web analytic applications. It runs on top of **flask plotly.js and react.js**. Dash is well suited for building data visualization apps with highly customized user interfaces.
- Panel **works with visualizations** from Bokeh, Matplotlib, HoloViews, and many other Python plotting libraries, making them instantly viewable, either individually or when combined with interactive widgets that control them. Panel works equally well in **Jupyter** Notebooks for creating quick data exploration tools or as a **standalone** deployed app in dashboards and allows you to **easily switch between those contexts** as needed.
- Voila **turns Jupyter notebooks into standalone web applications**. It's compatible with separate layout tools like Jupyter-flex or templates like voila-beautify.
- Streamlit can easily turn data scripts into shareable web apps with three main principles: embrace Python scripting, **treat widgets as variables**, and reuse data and computation.

Dashboard tools

bokeh

Bowtie

matplotlib

ipywidgets

Flask
web development,
one drop at a time

There are other tools that can be used for dashboarding.

- Bokeh is a plotting library, widget, and app library. It acts as a server for both plots and dashboards. Panel, which is one of the web-based dashboarding tools, is built on Bokeh.
- Ipywidgets provides a wide array of **Jupyter compatible widgets** and an interface supported by many Python libraries. But sharing as a dashboard requires a separate deployable server like voila.
- Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.
- Bowtie allows users to **build dashboards in pure Python**.
- Flask is a Python-backed **web server** that builds arbitrary websites, including those with Python plots that then function as Flask dashboards.

Recap

In this video, you learned that:

- Dashboard simplifies the dynamic aspects of the business.
- Data can be presented by using different types of dashboards.
- There are different types of dashboarding tools.

Additional Resources for Dashboards

- <https://pyviz.org/dashboarding/>

- <https://www.theguardian.com/news/datablog/2013/mar/15/john-snow-cholera-map>

Introduction to Plotly

What you will learn



Explore Plotly and its two sub-modules



Discover how to use Plotly graph objects and Plotly express for creating charts

What is Plotly?

Plotly is an **interactive**, open-source plotting library that supports over **40** unique chart types. It's available in **Python** and **JavaScript**.

Plotly **Python is an extension of Plotly JavaScript Library** and includes chart types like statistical, financial, maps, scientific, and three-dimensional data.

The **web-based visualizations** created using Plotly Python can be displayed in **Jupyter Notebook**, saved to standalone **HTML files**, or served as part of pure Python build web applications using dash.

Here, we'll be focusing on the **two sub-modules** of Plotly; Plotly Graph Objects and Plotly Express.

Plotly Graph Objects is the **low-level interface** to figures, traces and layout. Provides an automatically generated **hierarchy of classes**, figures, traces, and layout **called graph objects** that are used for representing figures with a **top-level class**: `Plotly.graph_objects.Figure`.

Plotly Express is a **high level wrapper** for Plotly. It's a recommended starting point for creating the most common figures provided by Plotly. Because of its simple syntax, **it uses graph objects internally**.

Using plotly.graph_objects

```
# Import required packages
import plotly.graph_objects as go
import plotly.express as px
import numpy as np

# Set random seed for reproducibility
np.random.seed(10)
x = np.arange(12)
# Create random y values
y = np.random.randint(50, 500, size=12)
```

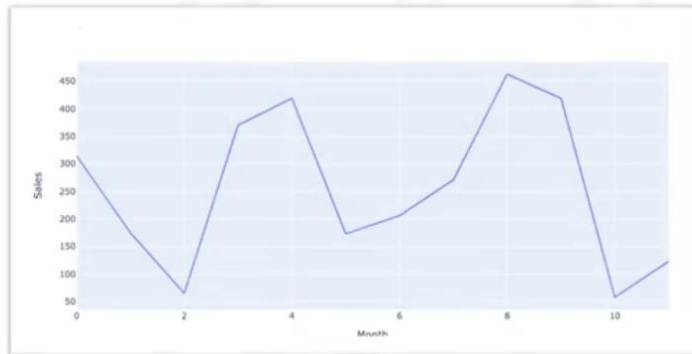
The plotly.graph contains **the JSON object**, which has a **dictionary structure**. Since we imported Plotly Graph Objects as go in the previous slide, go is the JSON object. The chart can be **plotted** by **updating** the **values** of the go object **keywords**.

We create the figure by adding a trace which is called Scatter.

Next, the layout of the figure is updated using the update_layout() method. Here we are updating the x-axis, y-axis, and chart title.

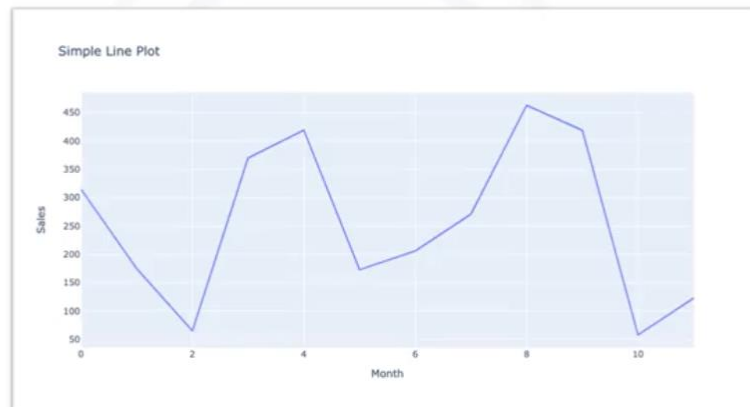
Using plotly.graph_objects

```
fig = go.Figure(data=go.Scatter(x=x, y=y))  
fig.update_layout(title='Simple Line Plot', xaxis_title='Month', yaxis_title='Sales')  
fig.show()
```



Using plotly.express

```
# Entire line chart can be created in a single command  
fig = px.line(x=x, y=y, title='Simple Line Plot', labels=dict(x='Month', y='Sales'))  
fig.show()
```



As you can see in the example, the entire line chart can be created using a single command.

It's time to play with the Plotly library. We'll use the airline reporting dataset from the data asset exchange to demonstrate how to use Plotly Graph Objects and express to create charts.

Here's a quick overview of the airline reporting dataset. The reporting carrier on-time performance dataset contains information on approximately 200 million domestic US flights reported to the United States Bureau of Transportation Statistics.

The dataset contains basic information about each flight, such as date, time, departure airport, and arrival airport, and if applicable, the length of time the flight was delayed and information about the reason for the delay.

Recap

In this video, you learned that:

- Plotly is an interactive, open-source plotting library that supports over 40 unique chart types.
- Plotly graph objects is a low-level interface to figures, traces, and layout.
- Plotly express is a high-level wrapper for Plotly. It uses graph objects internally.

Additional Resources for Plotly

<https://plotly.com/python/getting-started/>

[Plotly graph objects with example](#)

[Plotly express/](#)

[API reference](#)

[Plotly js cheat sheet.pdf](#)

[Plotly community](#)

[Related Blogs](#)

[IBM developer Data Asset Exchange \(DAX\)](#)

Plotly Basics: Scatter, Line, Bar, Bubble, Histogram, Pie, Sunburst

piplite is a special library used within **JupyterLite** to install Python packages.

```
import piplite
```

```
await piplite.install(['nbformat', 'plotly'])
```


Secuencia:

```
fig=go.figure()
```

```
fig.add_trace()
```

```
fig.update_layout()
```

```
fig.show()
```

```
fig.add_trace(go.Scatter(x=age_array, y=income_array, mode='markers', marker=dict(color='blue')))
```

```
fig.add_trace(go.Scatter(x=mon_array, y=num_array, mode='lines', marker=dict(color='green')))
```

Practice Quiz: Creating Dashboards with Plotly

Lesson 2: Working with Dash

Introduction to Dash

What you will learn



Explain what Dash is
and its uses



Describe what Dash
HTML components
are

Dash is an open-source, user interface Python library for creating **reactive web-based applications**. It's both enterprise-ready and a first-class member of Plotly's open-source tools.

Dash applications are web servers running Flask and **communicating JSON packets over HTTP requests**. Dash's **front end** renders components using **React.js**.

It's easy to build graphical user interfaces using Dash as it abstracts all technologies required to make the applications.

Dash is **declarative and reactive**. It can be rendered in a web browser and deployed to servers.

It provides a simple reactive decorator for binding code to UI. They are inherently **mobile and cross-platform-ready**.

Let's say you plan to create an application to answer business questions. As a first step, you need to determine the **application's layout**, decide which chart to use, and where to place it. For example, this is called the layout part of Dash. The second part is to add **interactivity** to the application.

Dash – An Overview

- Open-source User Interface Python library from Plotly
- Easy to build GUI
- Declarative and Reactive
- Rendered in a web browser and can be deployed to servers
- Inherently cross-platform and mobile ready

There are two components of Dash.

Dash Components

- Core components

```
import dash_core_components as dcc
```

- HTML Components

```
import dash_html_components as html
```

Let's see them one by one.

The **dash_core_components** describe higher-level **interactive components** generated with **JavaScript, HTML, and CSS through the React.js** library. Some examples of core components are creating a slider, input area, check items, and date picker.

Core Components

- Higher-level components that are interactive and are generated with JavaScript, HTML, and CSS through the React.js library
- Example: Creating a slider, input area, check items, datepicker, and other components

The `dash_HTML_components` library has **a component for every HTML tag**. You can compose your layout using Python structures with the `Dash_HTML` Dash components library.

The `dash_HTML_components` library provides classes for all HTML tags and the keyword arguments describe the HTML attributes like style, class name, and ID. No knowledge of HTML or CSS is required, but it can help style the dashboards.

HTML Components

- Components for every HTML tag
- Keyword arguments describe the HTML attributes like style, className, and id

Recap

In this video, you learned that:

- Dash is an Open-Source User Interface Python library for creating reactive, web-based applications.
- It is easy to build Graphical User Interfaces using Dash as it abstracts all technologies required to make the applications.
- There are two components of Dash: Core and HTML components.
- The `dash_core_components` describe higher-level interactive components generated with JavaScript, HTML, and CSS through the React.js library.
- The `dash_html_components` library has a component for every HTML tag.

Overview of Cloud IDE lab environment

Dash Basics: HTML and Core Components

```
python3.8 -m pip install packaging  
python3.8 -m pip install pandas dash  
pip3 install httpx==0.20 dash Plotly
```

Additional Resources for Dash

[Dash Python User Guide](#)

[Dash Core Components](#)

[Dash HTML components](#)

[Dash Community forum](#)

Make Dashboards Interactive

What you will learn



Describe the
Callback function



Determine how to
connect core and
HTML components
using Callbacks

Let us understand how to connect Core and html components using Callbacks.

Dash: Callbacks

- The Callback function is a python function
- It is automatically called by Dash

Decorator

`@app.callback`

A **callback function** is a Python **function** that is **automatically called** by Dash **whenever** an **input component's property changes**. The callback function is **decorated** with **@app.callback** decorator. So what does this decorator tell Dash? Basically, whenever there's a change in the input component value, the callback function wrapped by the decorator is **called followed by the update to the output component children** in the application layout.

Let's look at the callback function skeleton.

First, create a function that will perform operations to return the desired result for the output component.

```
def callback_function:
    .....
    .....
    .....
    return some_result
```

Decorate the callback function with @APP.callback decorator.

`@app.callback(Output, Input)`

This takes two parameters.

Output: set results returned from the callback function to a **component id**.

Input: set input is provided to the callback function to a component id.

From here, we **connect input and output to a desired property**.

We will see this in action with an example using the airline data. Use the case here to extract the top ten airline carriers in the provided input year. In terms of the number of flights **based on the input year, the output will change**.

Callback with one input

```
# Import required packages
import pandas as pd
import plotly.express as px
import dash
import dash_html_components as html
import dash_core_components as dcc
from dash.dependencies import Input, Output

# Read the data
airline_data = pd.read_csv('airline_2m.csv',
                           encoding = "ISO-8859-1",
                           dtype={'Div1Airport': str,
                                   'Div1TailNum': str,
                                   'Div2Airport': str,
                                   'Div2TailNum': str})
```

The new entry here is Dash dependencies, we're **importing Input and Output** that we will use in the callback function.

We load our data frame at the start of the app and it can be read inside the callback function.

Callback with one input

```
app = dash.Dash()
# Design dash app layout
app.layout = html.Div(children=[html.H1('Airline Dashboard',
                                         style={'textAlign': 'center', 'color': colors['text'],
                                                'font-size': 40}),
```

We will start **designing** the Dash application layout by **adding components**.

First, we'll provide the title to the Dash app using the html heading component H1 and style it using the style parameter.

Next, we're adding an html division and text Input core component.

```
html.Div(["Input: ", dcc.Input(id='input-yr', value='2010',
type='number', style={'height': '50px', 'font-size': 35}),],
style={'font-size': 40}),
```

In dash.Dash() The inputs and outputs of the application are simply the **properties of a particular component**.

In this example, our **input is the value property of the component that has the id input-yr**. and the default value 2010. We will update this value in our callback function.

Lastly, we will add a division with a graph core component. The core component has bar plot as id, which we will update inside the callback function.

Callback with one input

```
app = dash.Dash()
# Design dash app layout
app.layout = html.Div(children=[ html.H1('Airline Dashboard',
style={'textAlign': 'center', 'color': colors['text'],
'font-size': 40}),
html.Div(["Input: ", dcc.Input(id='input-yr', value='2010',
type='number', style={'height': '50px', 'font-size': 35}),],
style={'font-size': 40}),
html.Br(),
html.Br(),
html.Div(dcc.Graph(id='bar-plot')),
])
```

Note the component IDs.

```
@app.callback( Output(component_id='bar-plot', component_property='figure'),
               Input(component_id='input-yr', component_property='value'))
```

We will add a callback decorator `@app.callback`. **Input** to the callback will be the **component with id input-yr and property value**. **Output** to the callback will be the component id, bar-plot and **property figure**.

Component_id and component_property keywords are optional and provide clarity.

Next, we will define the callback function `get_graph`.


```
def get_graph(entered_year):
    # Select data
    df = airline_data[airline_data['Year']==int(entered_year)]
    # Top 10 airline carrier in terms of number of flights
    g1 = df.groupby(['Reporting_Airline'])['Flights'].sum().nlargest(10).reset_index()
    # Plot the graph
    fig1 = px.bar(g1, x='Reporting_Airline', y='Flights', title='Top 10 airline carrier in
        year ' + str(entered_year) + ' in terms of number of flights')
    fig1.update_layout()
    return fig1
```

The **entered year will be the input**. Using the year, we extract the required information from data and the application layout graph updates.

Lastly, we will run the application.

```
if __name__ == '__main__':
    app.run_server(port=8002, host='127.0.0.1', debug=True)
```

This is the output of the code our initial input year is 2010.



As the year is updated, the graph is getting updated in parallel.

The second example is a callback with two inputs.

Callback with two inputs

```
app = dash.Dash()
# Design dash app layout
app.layout = html.Div(children=[ html.H1('Airline Dashboard', style={'textAlign': 'center',
                                'color': colors['text'], 'font-size': 40}),
                                html.Div(["Year: ", dcc.Input(id='input-yr', value='2010',
                                type='number', style={'height': '50px', 'font-size': 35}),
                                ], style={'font-size': 40}),
                                html.Div(["State Abbreviation: ", dcc.Input(id='input-ab',
                                value='AL', type='text', style={'height': '50px',
                                'font-size': 35})], style={'font-size': 40}),
                                html.Br(),
                                html.Br(),
                                html.Div(dcc.Graph(id='bar-plot')),
                                ])

```

It's similar to one input callback except for a few changes. We will **add a division with one more text input** with the component id input-ab.

Then **add the new input** with component id input-ab to the decorator **inside a list**.

```
@app.callback( Output(component_id='bar-plot', component_property='figure'),
               [ Input(component_id='input-yr', component_property='value'),
                 Input(component_id='input-ab', component_property='value')])

```

Next we'll define callback function get_graph.

```
def get_graph(entered_year, entered_state):
    # Select data
    df = airline_data[ (airline_data['Year']==int(entered_year)) &
                      (airline_data['OriginState'] == entered_state)]

    # Top 10 airline carrier in terms of number of flights
    .....
    fig1.update_layout()
    return fig1

```

This takes the entered year and the entered state as input parameters. Computation extracts the information and the application layout updates with the graph.

This is the output of the code.



Our initial input year is 2010 and the state is AL, which is Alabama. As the year and state update, you'll observe that the graph gets updated in parallel.

Recap

In this video, you learned that:

- A callback function is a python function that is automatically called by Dash.
- The `@app.callback` decorator decorates the callback function in order to tell Dash to call it whenever there is a change in the input component value.
- The callback function takes input and output components as parameters and performs operations to return the desired result for the output component.

Additional Resources for Interactive Dashboards

[Python decorators reference 1](#)

[Python decorators reference 2](#)

[Callbacks with Examples](#)

[Dash app Gallery](#)

[Dash Community Components](#)

Add Interactivity: User Inputs and Callbacks

What you will learn



Describe the Skills
Network Labs
Cloud IDE
environment



Analyze flight on-
time data by
creating line
graphs with
dashboard
components



Create the layout of
the dash
application



List the steps to
run and launch the
application

Explain about an IDE

Create a skeleton for dash application

```
# Create a dash application  
app = dash.Dash(__name__)
```

Now it's time to design the layout of the app.

One, we will first create the dash app by calling the dash function, app equals dash.dash function. By passing `__name__`, you are telling the Dash application: "My files are located in the same directory as this script." This allows Dash to automatically determine the root path of your application and serve these static resources correctly.

Create a skeleton for dash application

```
# Build dash app layout
app.layout = html.Div(children=[ html.H1(),
                                html.Div(["Input Year: ", dcc.Input()],
                                      style={'font-size': 30}),
                                html.Br(),
                                html.Br(),
                                html.Div([
                                    html.Div(),
                                    html.Div()
                                ], style={'display': 'flex'}),
                                html.Div([
                                    html.Div(),
                                    html.Div()
                                ], style={'display': 'flex'}),
                                html.Div(, style={'width': '65%'})
                                ])
```

Two, to add a layout division using HTML, you can utilize the dcc.input function tag inside the layout division along with the div function and input components.

Three, using the display as flex for two outer divisions to get graphs side-by-side in a row.

Four, right or copy the code from the instructions and paste it into the script file below the previous code.

Now that we have the skeleton, let's include the components.

Include title and input

```
html.H1('Flight Delay Time Statistics',
        style={'textAlign': 'center', 'color':
        '#503D36',
        'font-size': 30})
```

```
html.Div(["Input Year: ", dcc.Input(id='input-year', value='2010',
                                    type='number', style={'height': '35px', 'font-
size': 30})],),
```

One, the title will go in the html.H1 function as a string, and you can style the text.

Two, inside the first HTML division, which we refer to as the dcc.input tag, the input component can be customized with various attributes. For instance, you can assign the ID as input year, set the value as 2010, specify the type as number, and even customize the style attributes such as height and font size.

Include the graphs

```
html.Div([
    html.Div(dcc.Graph(id='carrier-plot')),
    html.Div(dcc.Graph(id='weather-plot'))
], style={'display': 'flex'})
```

One, when utilizing this dial parameter, you can actively add different graphs to the divisions. We assign display as flex to place two plots side-by-side in a row. Use dcc.graph tag for including the plot and give each plot an ID.

Prepare data to plot

```
def compute_info(airline_data, entered_year):
    # Select data
    df = airline_data[airline_data['Year']==int(entered_year)]
    # Compute delay averages
    avg_car =
df.groupby(['Month', 'Reporting_Airline'])['CarrierDelay'].mean().reset_index()
    avg_weather =
df.groupby(['Month', 'Reporting_Airline'])['WeatherDelay'].mean().reset_index()
    avg_NAS =
df.groupby(['Month', 'Reporting_Airline'])['NASDelay'].mean().reset_index()
    avg_sec =
df.groupby(['Month', 'Reporting_Airline'])['SecurityDelay'].mean().reset_index()
    avg_late =
df.groupby(['Month', 'Reporting_Airline'])['LateAircraftDelay'].mean().reset_index()
    return avg_car, avg_weather, avg_NAS, avg_sec, avg_late
```

For the year input by user, we need to compute the airline data and make charts and plots. One, let's call this function compute_info. Pass the dataset and the input year as arguments, the function should return all the average delays for that year. Two, within this function, we will aggregate the specific delay by grouping the dataset for the month and the reporting airlines features. You can compute all the delays as individual plots. But best practice would be to create a function.

As we need our dashboard to update in real time based on the year value input by the user, we will now create a callback function.

First, we will create a callback decorator.

Callback for real-time dashboard update

```
@app.callback( [
    Output(component_id='carrier-plot', component_property='figure'),

    Input(component_id='input-year', component_property='value')

def get_graph(entered_year):

    # Compute required information for creating graph from the data
    avg_car, avg_weather, avg_NAS, avg_sec, avg_late =
compute_info(airline_data, entered_year)

    # Line plot for carrier delay
    carrier_fig = px.line(avg_car, x='Month', y='CarrierDelay',
color='Reporting_Airline', title='Average

if __name__ == '__main__':
    app.run_server()
```

One, update the output component ID parameter with the IDs provided in the dcc.graph function component **for all five** components we created and set the component property as a figure, like this.

Two, and then update the input component ID parameter with the ID and set the property as value.

Three, then create a callback function that uses the input provided to perform the computation.

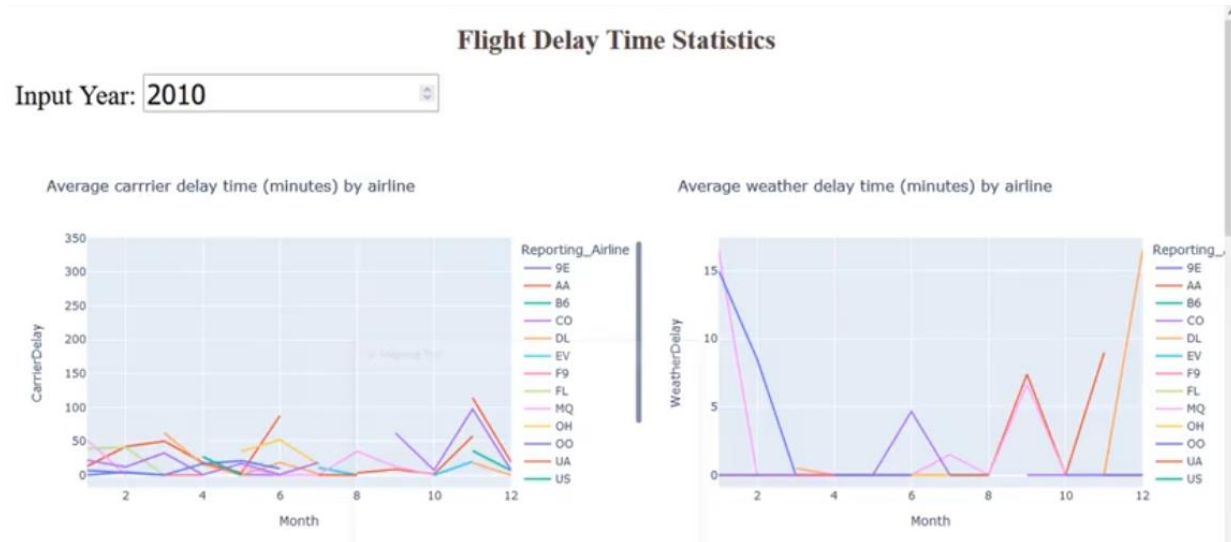
Four, create a graph in this function and return it as an output.

Five, lastly, call the run.server function on the app.

Six, good work.

Seven, to run this application, use the command `python3 flight-delay.py` in the terminal.

This is the aim



Recap

In this video, you learned

- We have walked through the lab where you developed a dashboard with Dash framework on Flight Delay Time Statistics

Flight Delay Time Statistics Dashboard

After completing the lab you will be able to:

- Know how to add multiple graphs to the dashboard
- Work with Dash Callbacks to handle multiple outputs
-

It utilizes [Theia](#), an open-source IDE platform, that can be run on desktop or on the cloud.

Dashboard Components

- Monthly average carrier delay by reporting airline for the given year.
- Monthly average weather delay by reporting airline for the given year.
- Monthly average national air system delay by reporting airline for the given year.
- Monthly average security delay by reporting airline for the given year.
- Monthly average late aircraft delay by reporting airline for the given year.

NOTE: Year range should be between 2010 and 2020

```
python3.8 -m pip install packaging
```

```
python3.8 -m pip install pandas dash
```

```
pip3 install httpx==0.20 dash Plotly
```

The primary reason for the second command is to install the other, essential dependencies (httpx and plotly) that you didn't include in the first command.

Practice Quiz: Working with Dash

Module 4 Summary: Creating Dashboards with Plotly and Dash

- Dash is an Open-Source User Interface Python library for creating **reactive, web-based applications**.
- It is easy to build Graphical User Interfaces using Dash as it **abstracts all technologies** required to make the applications.
- There are two components of Dash: **Core and HTML components**.
- The dash_**core**_components describe higher-level **interactive components** generated with JavaScript, HTML, and CSS through the React.js library.
- The dash_**html**_components library has a **component for every HTML** tag.
- A callback function is a python function that is automatically called by Dash **whenever an input component's property changes**.
- The @app.callback decorator decorates the callback function in order to tell Dash to call it whenever there is a change in the input component value.
- The callback function takes **input and output components as parameters** and performs operations to return the desired result for the output component.

Module 4 Cheat Sheet

Module 4 Graded Quiz: Creating Dashboards with Plotly and Dash

Module 5

Title: Final Project and Exam

Description

The primary focus of this module is to practice the skills gained earlier in the course and then demonstrate those skills in your final assignment. For the final assignment you will analyze historical automobile sales data covering periods of recession and non-recession. You will bring your analysis to life using visualization techniques and then display the plots and graphs on dashboards. Finally, you will submit your assignment for peer review and you will review an assignment from one of your peers. To wrap up the course you will take a final exam in the form of a timed quiz.

Objectives

By the end of this week, you will be able to:

- Practice visualization skills
- Practice creating a dashboard
- Create various visualizations using a number of plot libraries
- Create a dashboard and add interactivity
- Review and grade an assignment submitted by peers

Activities

Lesson 1: Practice Project

Practice Project Overview

Practice Assignment: Part 1 - Analyzing wildfire data in Australia

Practice Assignment: Part 2 - Creating Dashboards

Lesson 2: Final Project

Final Project Overview

Common Issued in Dash Application

Key considerations for developing a Dash application

1. Please ensure you have added the correct column values as indicated in the hint for each code snippet. Example for **X and Y** in the charts
2. Please ensure that you have used the correct **indentation** in the code.

3. Fill in your code using the **placeholders(...)** provided in the code skeleton provided in the lab. Separate each component by commas.

Example: This creates a **Div element in HTML**, a container for other components. Inside this div, various child elements are placed, which need to be separated by commas.

```
html.Div([
    html.Label("Select Statistics:"),
    dcc.Dropdown(
        id='.....',
        options=.....,
        value='.....',
        placeholder='.....'
    )
]),
```

Common errors:

1. *Syntax error: invalid syntax def update_input_container(selected_statistics):*

Solution: Check if any parentheses or square brackets are unclosed before the **def** function.

The HTML components are organized in a hierarchy using brackets. The use of square brackets **[] within html.Div()** or any other Dash component is essential for defining children elements in a list. This list-based structure allows you to include multiple child components inside a single-parent component.

Outer HTML

```
html.Div([ ...
    ...
])
```

This `html.Div` is the container for the entire layout of the dashboard. It encapsulates and structures all the other HTML elements as a cohesive unit.

They use square brackets to group multiple child components within a single-parent component. This enables you to structure complex layouts with nested elements.

Dash components need to maintain a clear hierarchy to render correctly. The square brackets help maintain this hierarchy by clearly indicating which components are children of a particular parent component.

Inner HTML

```
html.Div([
    html.Label("Select Statistics:"),
    dcc.Dropdown(
        id='.....',
        options=.....,
        value='.....',
        placeholder='.....'
    )
]),
```

2. When encountering a Callback error updating output. *container.children*

Solution: Ensure **Input variables** used within the **callbacks** are defined and correctly scoped in the correct order.

```
@app.callback(
    Output(component_id='output-container', component_property='children'),
    [Input(component_id='dropdown-statistics', component_property='value'),
    Input(component_id='select-year', component_property='value')])
```

3. *Graph not displaying.*

Solution: Ensure you have used the **elif** statement with the correct variable.

```
elif (input_year and selected_statistics=='.....') :
    yearly_data = data[data['Year'] == .....]
```

Final Assignment: Part 1 - Create Visualizations using Matplotlib, Seaborn & Folium

Final Assignment: Part 2 - Create Dashboard with Plotly and Dash

Final Assignment: Part 3 - Submission and Grading

Final Exam: Data Visualization with Python - Timed Quiz

Lesson 3: Course Wrap Up

Course Summary

Congratulations and Next Steps

Thanks from the Course Team